Graph Data Modeling for Neo4j

Course Guide

Table of Contents

Graph Data Modeling for Neo4j . . . 1
- About this course . . . 1
- Lesson Overview . . . 1
- Resources . . . 1

Introduction to Graph Data Modeling . . . 2
- About this module . . . 2
- What is graph data modeling? . . . 2
- How does Neo4j support graph data modeling? . . . 2
  - Neo4j Property Graph Model . . . 2
  - Traversal in the graph . . . 3
- Tools for graph data modeling . . . 4
- Guided Exercise: Using the Arrow tool . . . 4
- Workflow for graph data modeling . . . 4
- Check your understanding . . . 5
  - Question 1 . . . 5
  - Question 2 . . . 5
  - Question 3 . . . 5
- Summary . . . 6

Designing the Initial Graph Data Model . . . 7
- About this module . . . 7
- Designing the initial data model . . . 7
- Step 1: Understanding the domain . . . 7
  - Example domain: Bill of Materials . . . 8
  - Example BOM use cases . . . 8
- Step 2: Create high-level sample data . . . 9
  - BOM high-level sample data . . . 9
- Step 3: Define specific questions for the application . . . 10
  - Sample questions for the BOM . . . 10
- Step 4 : Identify entities . . . 10
  - Identify entities from the questions . . . 10
  - Define properties . . . 11
- Exercise 1: Identifying entities for the BOM application . . . 11
  - Exercise 1 solution . . . 12
- Exercise 2: Creating the BOM entity model in the Arrow tool . . . 12
  - Exercise 2 solution . . . 12
- Step 5: Identify connections between entities . . . 13
  - Identify connections between entities . . . 14
  - Naming relationships . . . 14

Direction and type . . . 15
How much fanout will a node have? . . . 15
Exercise 3: Adding relationships to the model . . . 16
Exercise 3 instructions . . . 16
Exercise 3 solution . . . 17
Example: Detailed sample data for the BOM application . . . 18
Step 6: Test the questions against the model . . . 21
Testing the model - 1 . . . 21
Testing the model - 2 . . . 22
Testing the model - 3 . . . 23
Testing the model - 4 . . . 23
Step 7: Test scalability . . . 24
Testing scalability . . . 25
Check your understanding . . . 25
Question 1 . . . 25
Question 2 . . . 26
Question 3 . . . 26
Summary . . . 26
Graph Data Modeling Core Principles . . . 27
About this module . . . 27
Best practices for modeling nodes . . . 27
Uniqueness of nodes: before . . . 27
Uniqueness of nodes: after . . . 28
Complex data . . . 28
Use fanout judiciously for complex data . . . 29
Best practices for modeling relationships . . . 30
Using specific relationship types . . . 30
But… not too specific . . . 30
No symmetric relationships . . . 31
Semantics of symmetry are important . . . 32
Using types vs. properties . . . 32
Property best practices . . . 33
Data accessibility . . . 33
Hierarchy of accessibility . . . 34
Check your understanding . . . 35
Question 1 . . . 35
Question 2 . . . 35
Question 3 . . . 36
Summary . . . 36
Common Graph Structures . . . 37
About this module . . . 37

Intermediate nodes . . . 37
Using intermediate nodes . . . 37
Intermediate nodes: sharing context . . . 38
Intermediate nodes: sharing data . . . 39
Intermediate nodes: organizing data . . . 40
Linked lists . . . 40
Interleaved linked list . . . 41
Head and tail of linked list . . . 41
Timeline tree . . . 42
Using multiple structures . . . 43
Using the timeline tree . . . 44
Using intermediate nodes . . . 44
Using linked list . . . 44
Exercise 4: Model a Learning Management System (LMS) . . . 45
Exercise 4 instructions: The domain . . . 45
Exercise 4 instructions: Sample data . . . 45
Exercise 4 instructions: Application questions . . . 46
Exercise 4 solution: Application model . . . 46
Exercise 4 solution: Sample data . . . 48
Check your understanding . . . 51
Question 1 . . . 52
Question 2 . . . 52
Question 3 . . . 52
Summary . . . 52
Refactoring and Evolving a Model . . . 53
About this module . . . 53
What is refactoring? . . . 53
Review: Hierarchy of Accessibility . . . 53
Why refactor? . . . 54
Goal: Eliminate duplicate data in properties . . . 55
Refactor example: Extracting nodes from properties . . . 55
Goal: Use labels instead of property values . . . 56
Refactor example: Turn property values into labels for nodes . . . 57
Goal: Use nodes instead of properties for relationships . . . 58
Refactor: Extract nodes from relationship properties . . . 59
Refactoring example: Modeling airline flights . . . 59
Initial question for our model . . . 60
Initial model . . . 60
Refactor: Create intermediate Flight nodes . . . 61
Refactor: Create AirportDay intermediate nodes . . . 61
Possible refactor: Change relationship type to date . . . 62

Possible refactor: Remove Airport nodes . . . 62
Refactor: Add Destination intermediate nodes . . . 63
Check your understanding . . . 64
Question 1 . . . 64
Question 2 . . . 65
Question 3 . . . 65
Summary . . . 65
Graph Data Modeling for Neo4j: Summary . . . 66
Course Summary . . . 66
Resources . . . 66
Course feedback . . . 67

Graph Data Modeling for Neo4j

About this course

This course introduces you to how to develop a graph data model for an application using Neo4j best practices.

This course is intended for developers and architects. You need not have programming experience to take this training, but you should have a solid understanding of what Neo4j is and how graph components are used in a Neo4j database.

In this course, you complete hands-on exercises to gain experience with developing graph data models for use with Neo4j.

To complete the hands-on exercises for this course, you need a Web browser and Internet access.

This Course is published by Neo4j per this License for Use.

Lesson Overview

Here are the lessons of this course:

1. Introduction to Graph Data Modeling
2. Designing the Initial Graph Data Model
3. Graph Data Modeling Core Principles
4. Common Graph Structures
5. Refactoring and Evolving a Graph Data Model

Resources

We have created a discussion area in our Neo4j Community Site, if you run into problems in the course and need assistance. You should register on the Community site where you can view other questions and answers for students taking our online training courses. The Neo4j Community Site is an excellent resource for answering many types of questions posed by other users of Neo4j.

Here are some resources you may use as you go through this course:

- Neo4j Cypher Manual
- Neo4j Developer Resources
- Neo4j Community Site

Introduction to Graph Data Modeling

About this module

At the end of this module, you should be able to:

- Describe what graph data modeling is.
- Describe how Neo4j supports graph data modeling.
- Describe the tools you can use for graph data modeling.
- Describe the workflow for graph data modeling.

What is graph data modeling?

Graph data modeling is a collaborative effort where the application domain is analyzed by stakeholders and developers to come up with the optimal model for use with Neo4j. Stakeholders must understand the domain and be prepared to ask detailed questions about how the business-at-hand operates.

Stakeholders are:

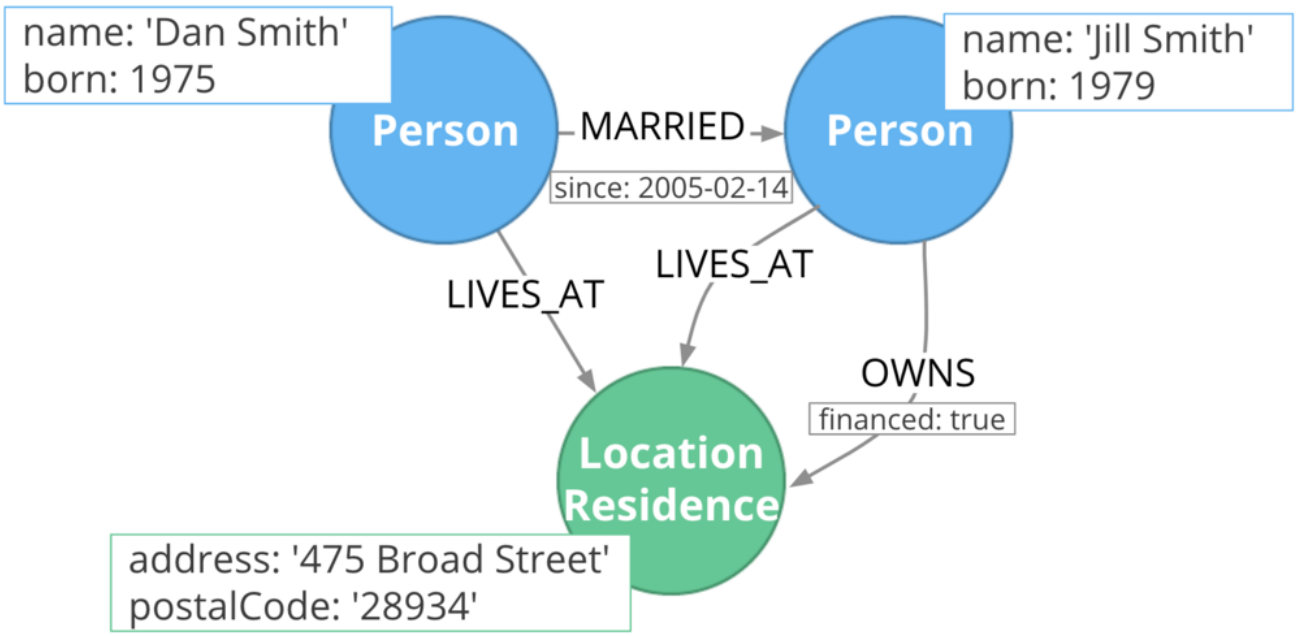
- Business analysts
- Architects
- Managers
- Project leaders

How does Neo4j support graph data modeling?

- Neo4j is a full-featured graph database that allows you to create **property graphs**.
- Applications retrieve data for business use cases by **traversing the graph**.

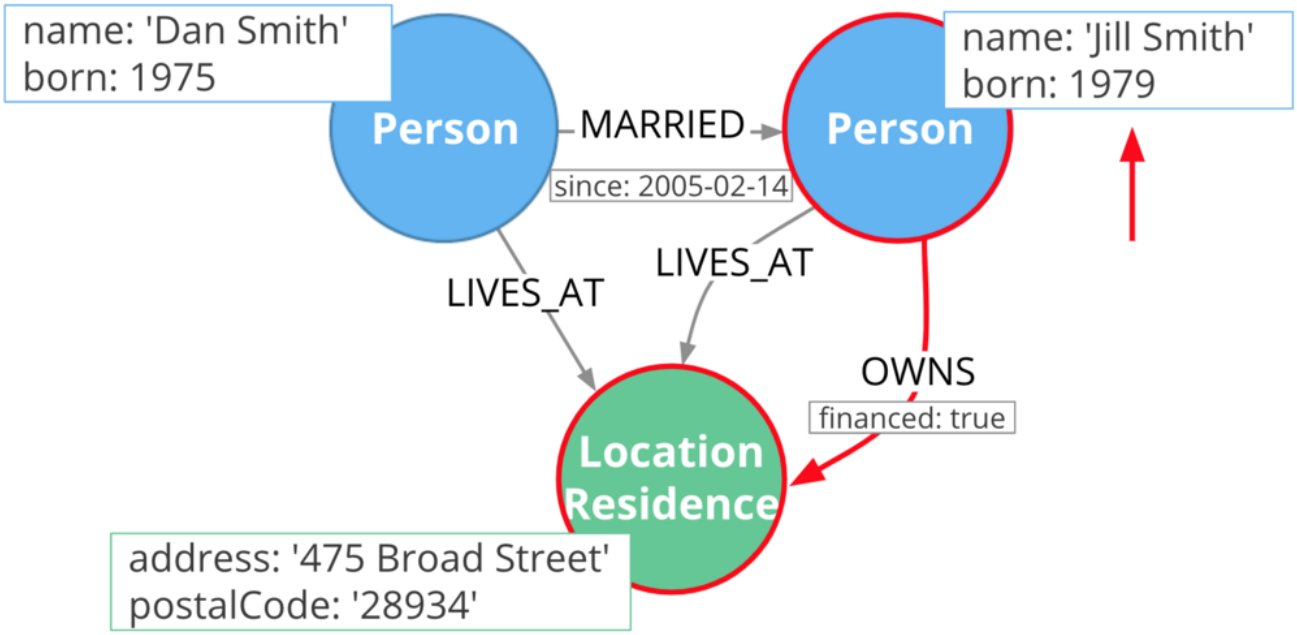
Neo4j Property Graph Model

- Nodes (Entities)
- Relationships
- Properties
- Labels



Nodes and relationships are the key components of a graph. Nodes should have labels to categorize entities. Relationships must have direction and type. Nodes and relationships can have properties. A label is used to categorize a set of nodes. Properties are used to provide specific values to a node or relatiobship.

Traversal in the graph

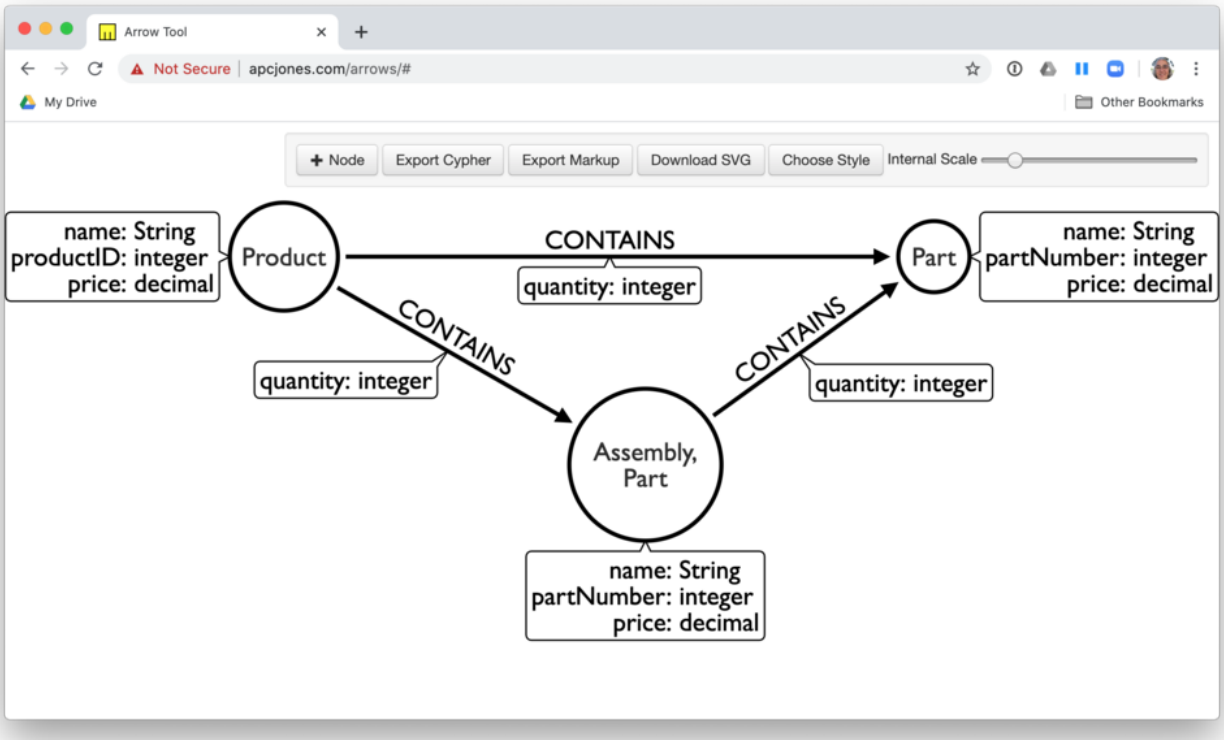



```
MATCH (r:Residence)<-[:OWNS]-(p:Person)
WHERE r.address = '475 Broad Street'
RETURN p
```

Traversal means anchoring a query based upon a property value, then traversing the graph to satisfy the query. In this example, the address property of the Residence node is used to determine where to anchor the query. Then, the *OWNS* relationship is traversed to retrieve the qualifying

nodes. A relationship is only traversed once in a query.

Tools for graph data modeling



There are many ways that developers and stakeholders can work together to build a graph data model for their application. Drawing it on a whiteboard may be done initially, but you will need to share your model with others so using a tool is recommended. The tool that we use in this course is the Arrow tool which integrates with Neo4j. With the Arrow tool, you can generate Cypher statements for creating code to implement the model.

Guided Exercise: Using the Arrow tool

Follow along with this video to learn how to begin using the Arrow tool for graph data modeling.

https://youtu.be/NB184T-S46w

Workflow for graph data modeling

Step	Description	Stakeholders	Developers
1.	Build the initial graph data model.	✔	✔
2.	Create and profile Cypher queries to support the model.		✔
3.	Create data in the database to support the model.		✔
4.	Identify additional questions for the application.	✔	✔
5.	Modify the graph data model to support new questions.		✔
6.	Refactor the database to support the revised graph data model.		✔
7.	Create and profile Cypher queries to support the revised model.		✔
	Repeat Steps 4-7.	✔	✔

Data model maintenance is cyclical and iterative. We are comfortable with this idea because updating a graph model is much cheaper and easier than updating the schema of a tabular store. Ease-of-updating will be a consistent theme throughout this training.

Check your understanding

Question 1

What elements of a Neo4j graph are used to categorize entities?

Select the correct answer.

- ☐ Relationship
- ☐ Property
- ☐ Node
- ☑ Label

Question 2

Which statements below are true about Neo4j graph traversal?

Select the correct answers.

- ☑ Traversal during a query starts at a set of anchor nodes.
- ☐ Traversal during a query ends at an anchor node.
- ☑ A relationship is only traversed once during a query.
- ☐ A relationship can be traversed multiple times during a query.

Question 3

What modeling tool is designed specifically for use with Neo4j?

Select the correct answer.

- ☐ Visio
- ☐ Sketch
- ☑ Arrow Tool
- ☐ Neo4j Modeler

Summary

You should now be able to:

- Describe what graph data modeling is.
- Describe how Neo4j supports graph data modeling.
- Describe the tools you can use for graph data modeling.
- Describe the workflow for graph data modeling.

Designing the Initial Graph Data Model

About this module

At the end of this module, you should be able to:

- Describe the domain for a model.
- Define the questions for the domain.
- Identify entities from the questions for the domain.
- Use the Arrow Tool to model the domain.
- Identify the connections between entities.
- Describe how to test the initial model.

Designing the initial data model

1. Understand the domain.
2. Create high-level sample data.
3. Define specific questions for the application.
4. Identify entities.
5. Identify connections between entities.
6. Test the questions against the model.
7. Test scalability.

Designing the initial model is a multi-step process. We find it is extremely important to be disciplined about this. If you do, you tend to get to a first model faster, that model better suits your documented needs, and that model is usually simple to understand. It will almost certainly not be perfect, and will need to change in response to shifting needs and a deepening understanding of the domain, use case, and capabilities of Neo4j. But recall that modifying a data remodel in Neo4j is relatively inexpensive. This fact reduces the pain and risk of taking a narrow approach to the first model.

Later in this training, you will learn how to identify weaknesses in a model, and how to improve the existing design.

NOTE	Every organization has their own standards for how requirements are documented. You will not be learning how to document requirements, but rather focus on the model itself.

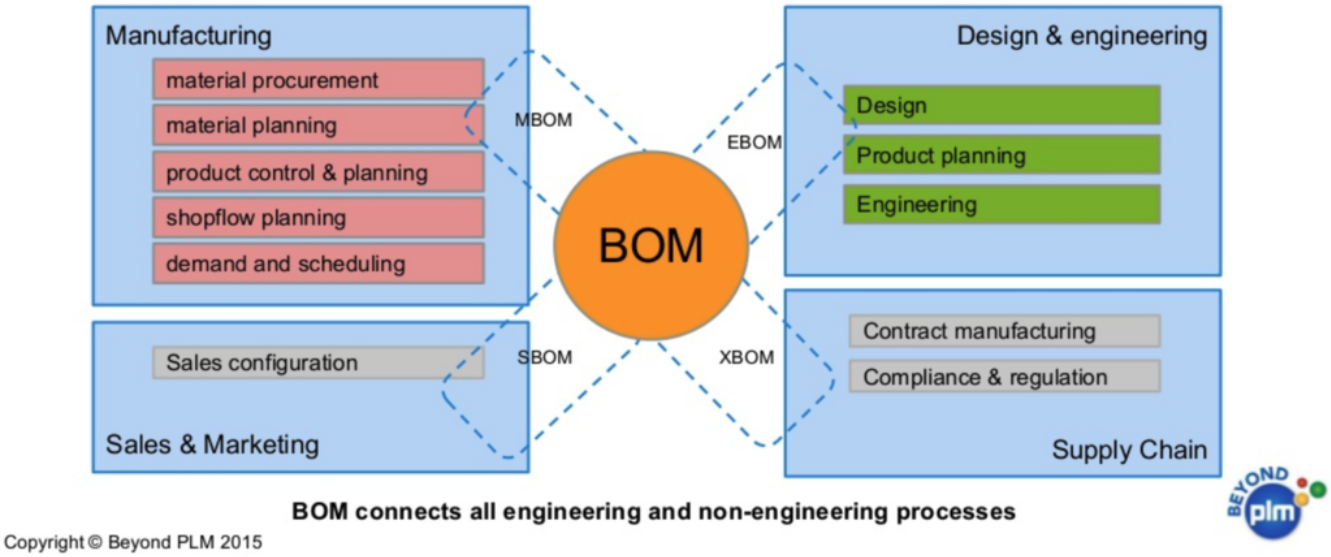
Step 1: Understanding the domain

- Describe the application in detail.
- Identify the stakeholders and developers of the application.

- Identify the users of the application (people, systems).
- Enumerate the use cases that are agreed upon by all stakeholders where users are part of the use case.

NOTE	No knowledge of the underlying implementation is required!

Example domain: Bill of Materials



From Wikipedia:

A bill of materials or product structure (sometimes bill of material, BOM or associated list) is a list of the raw materials, sub-assemblies, intermediate assemblies, sub-components, parts, and the quantities of each needed to manufacture an end product. A BOM may be used for communication between manufacturing partners or confined to a single manufacturing plant. A bill of materials is often tied to a production order whose issuance may generate reservations for components in the bill of materials that are in stock and requisitions for components that are not in stock.

A BOM can define products as they are designed (engineering bill of materials), as they are ordered (sales bill of materials), as they are built (manufacturing bill of materials), or as they are maintained (service bill of materials). The different types of BOMs depend on the business need and use for which they are intended. In process industries, the BOM is also known as the formula, recipe, or ingredients list. The phrase "bill of material" (or "BOM") is frequently used by engineers as an adjective to refer not to the literal bill, but to the current production configuration of a product, to distinguish it from modified or improved versions under study or in test.

Example BOM use cases

- System produces list of parts to make a product.
- System produces list of products that can be made with available parts.
- System produces list of parts that are made with other parts.
- User picks parts to make a product.
- System creates a price for a product based upon the part prices.

- System creates list of parts that need to be ordered.

NOTE	A product or part can be made of multiple parts of the same type. Some parts are made from other parts (sub-assembly).

This is a very small fraction of the use cases for the BOM application.

Step 2: Create high-level sample data

1. Understand the domain.
2. **Create high-level sample data.**
3. Define specific questions for the application.
4. Identify entities.
5. Identify connections between entities.
6. Test the questions against the model.
7. Test scalability.

Sample data need not be expansive, detailed, or even of particularly high quality. It simply needs to represent the broad shape of the information you would expect to find in the real data.

Sample data can either be completely made up, or can be a sample pulled from a real dataset. If you've done the work to properly understand the domain, it should not strongly matter either way.

BOM high-level sample data

Products	Parts	Assemblies	Notes
Wood table 40"	Wood top 40"	Leg assembly	Has 4 legs
Deluxe wood table 40"	Glass top 40"	Leg assembly	Has 4 legs
Wood table 60"	Wood top 60"	Leg assembly	Has 6 legs, table brace
Deluxe wood table 60"	Glass top 60"	Leg assembly	Has 6 legs, table brace
	Leg		
	Leg foot		
	M20 bolt		
	M20 nut		
	Leg plate		Uses 2 bolts/nuts per leg
	Table brace		

If you want your modeling process to be constructive, you must have enough data points for your domain so that you can understand the data that will be retrieved when the questions are asked.

The data is typically the "things" described in the use cases. You don’t include "system" or "user" as part of the data.

Step 3: Define specific questions for the application

1. Understand the domain.
2. Create high-level sample data.
3. **Define specific questions for the application.**
4. Identify entities.
5. Identify connections between entities.
6. Test the questions against the model.
7. Test scalability.

Sample questions for the BOM

1. What parts are needed to make Wood table 40"?
2. Do we have enough parts to make 100 Deluxe wood table 60"?
3. What products require a table brace?
4. How much will the parts cost to make product Wood table 60"?

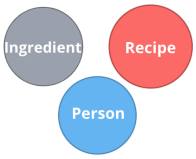
Step 4 : Identify entities

1. Understand the domain.
2. Create high-level sample data.
3. Define specific questions for the application.
4. **Identify entities.**
5. Identify connections between entities.
6. Test the questions against the model.
7. Test scalability.

Identify entities from the questions

Entities are the nouns in your application questions:

1. What ingredients are used in a recipe?
2. Who is married to this person?



- The generic nouns will often become labels in your model.
- Proper nouns will often become values for properties.
- Use domain knowledge to decide if entities need to be further grouped or differentiated.
- In Neo4j Enterprise Edition, there is no limit to the number entities (nodes) in the graph. (Community Edition has a limit of 34B)

As an example of using domain knowledge, you might happen to know that the distinction between wet and dry ingredients is important in your use case, or that vegetarian and vegan are important subgroups of a recipe. These may not appear clearly in the specific application questions.

Define properties

Properties serve one of two purposes:

1. Unique identification.
2. Answering one of the application questions.

Otherwise, they are merely "decoration".

Properties are used in a Cypher query for:

- Anchoring (where to begin the query).
- Traversing the graph (navigation).
- Returning data from the query.

Identifiers serve two purposes:

1. They can be used to power unique constraints or `MERGE` operations.
2. They can be useful to help a human user understand what they are looking at in query results.

There are subtly different best practices for properties serving each of those functions, which you will learn about later. For now, we simply group them all into this one generic bucket of usefulness.

Decorators should usually be left out of the initial model. If they are not being used to answer questions, they are a waste of storage, and their presence can distract users and developers from the information that is actually important. Remember that we endorse a narrow, focused approach to model design, based on the relative ease of modifying a graph data model.

However, it can be beneficial to include decorators when you move to production. Not because they will be used, but because, if all information is locally available to Neo4j, refactoring a model later may not require importing data from its original source.

Exercise 1: Identifying entities for the BOM application

Define the entities and properties from these questions:

1. What parts are needed to make Wood table 40"?
2. Do we have enough parts to make 100 Deluxe wood table 60"?
3. What products require a table brace?
4. How much will the parts cost to make product Wood table 60"?

Exercise 1 solution

Product

- name
- productId

Part

- name
- partNumber
- price

Part, Assembly

- name
- partNumber
- price

Assembly is the tricky one—it does not appear in the questions, and you must use domain knowledge to understand why it is needed.

Remember that your first model is never perfect, and that is OK because modifying the model in its early stages is relatively easy.

Exercise 2: Creating the BOM entity model in the Arrow tool

Use the entities you identified earlier for the BOM application and create them in the Arrow tool.

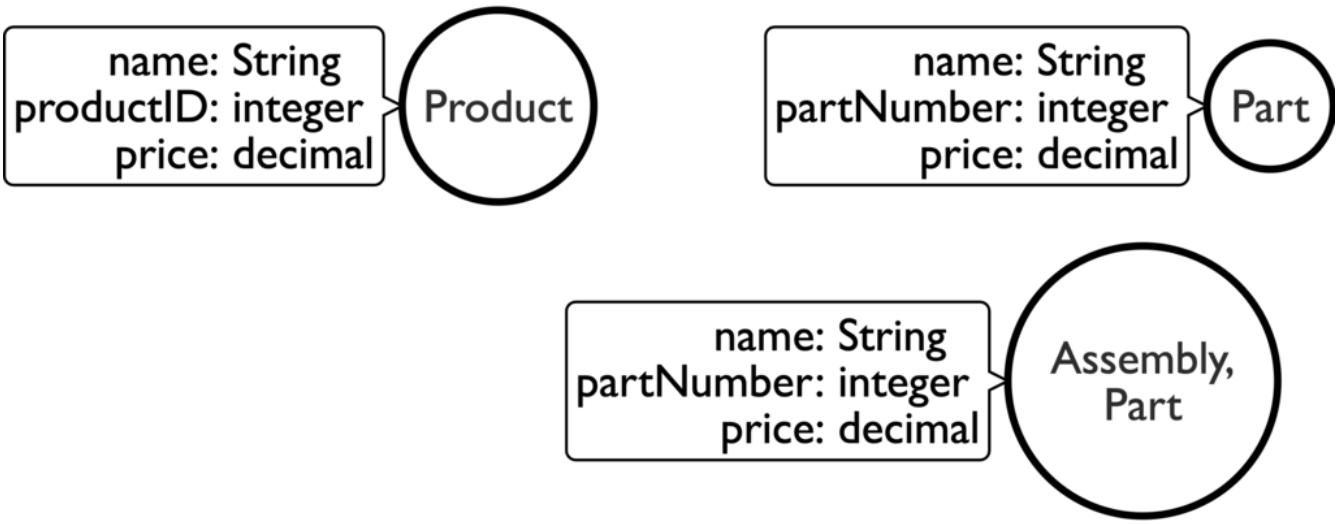
Make sure you include properties for the nodes and specify the types for the properties, rather than values.

Exercise 2 solution

Here is the Arrow tool markup for the solution:

```
<ul class="graph-diagram-markup" data-internal-scale="2.01" data-external-
scale="1">
  <li class="node" data-node-id="0" data-x="-106.54309068746238" data-y="-
40.13742460863777">
    <span class="caption">Part</span><dl
class="properties"><dt>name</dt><dd>String</dd><dt>partNumber</dt><dd>inte
ger</dd><dt>price</dt><dd>decimal</dd></dl></li>
  <li class="node" data-node-id="1" data-x="-262.3812669868761" data-y="-
40.13742460863777">
    <span class="caption">Product</span><dl
class="properties"><dt>name</dt><dd>String</dd><dt>productID</dt><dd>integ
er</dd><dt>price</dt><dd>decimal</dd></dl></li>
  <li class="node" data-node-id="2" data-x="-202.58837695145493" data-
y="24.371374691303693">
    <span class="caption">Assembly, Part</span><dl
class="properties"><dt>name</dt><dd>String</dd><dt>partNumber</dt><dd>inte
ger</dd><dt>price</dt><dd>decimal</dd></dl></li>
</ul>
```

Here is what it should look like in the UI of the Arrow tool:



Step 5: Identify connections between entities

1. Understand the domain.
2. Create high-level sample data.
3. Define specific questions for the application.
4. Identify entities.
5. **Identify connections between entities.**
6. Test the questions against the model.
7. Test scalability.

Identify connections between entities

Connections are the verbs in your application questions:

1. What ingredients are used in a recipe?



1. Who is married to this person?

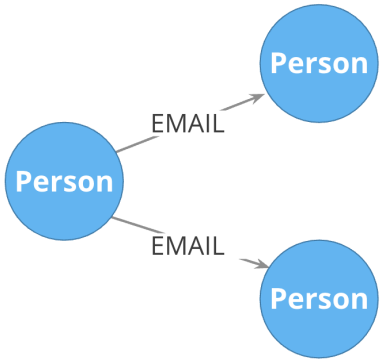


At a glance, connections are straightforward things, but their micro- and macro-design are arguably the most critical factors in graph performance. Using "connections are verbs" is a fine shorthand to get started, but there are other important considerations that will be discussed later in this training.

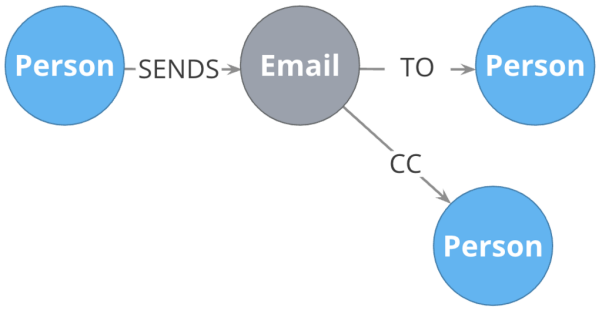
Naming relationships

- Stakeholders must agree upon name (type for the relationship).
- Avoid names that could be construed as nouns (for example email).
- Neo4j has a limit of 16M relationship types in Enterprise Edition (64K in Community Edition).

Do not do this:



Instead do this:



Choosing good names is vital. It needs to be something that is intuitive to stakeholders and developers alike, cannot be confused as an entity name, and is specific enough to make traversal efficient but general enough to make the Cypher easy to write. You will learn more about later in this training when you learn about the core principles of graph data modeling.

Direction and type

Direction and type are required in Neo4j.

Choose direction (and type) based on the expected questions:

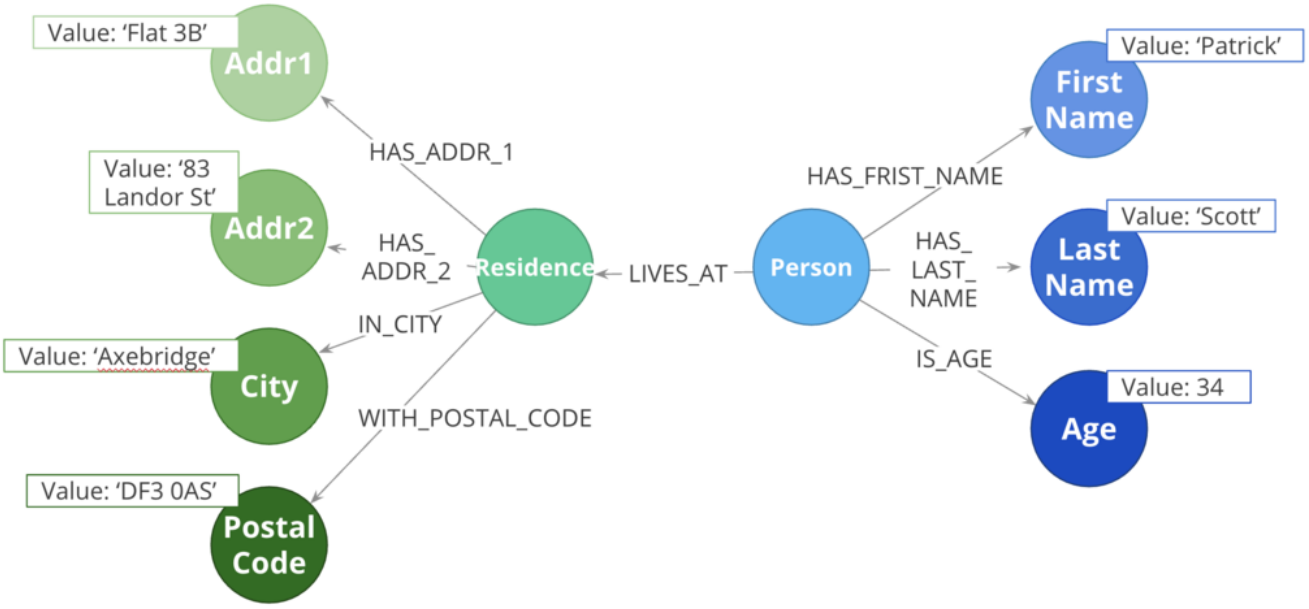
1. What episode follows The Ark in Space? (NEXT)
2. What episode came before Genesis of the Daleks? (PREVIOUS)



These are episodes of the Dr. Who BBC series, season 12, aired in 1975.

A relationship can be modeled in either direction, with no effect on performance. Naturally, the type would have to differ. Here, we have chosen left-to-right relationships of the type NEXT. We could equally well have chosen right-to-left relationships of the type PREVIOUS. This has no effect on query performance, but the Cypher used for each variant will differ. Choose whichever variant makes the Cypher more intuitive for developers to write, based on your expectations for what people will ask.

How much fanout will a node have?



Here, we have entities (Person, Address) represented not as a single node, but as a network or linked nodes. This is an extreme example of fanout, and is almost certainly overkill for any real-life solution, but some amount of fanout can be very useful.

For example, splitting last names onto separate nodes helps answer the question, "Who has the last name Scott?" Similarly, having cities as separate nodes assists with the question, "Who lives in the same city as Patrick Scott?"

The main risk about fanout is that it can lead to very dense nodes, or supernodes. Supernodes need to be handled carefully.

Exercise 3: Adding relationships to the model

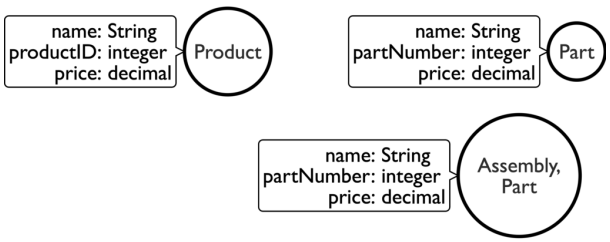
Exercise 3 instructions

Here are the questions we need to answer for our BOM application that you have already seen:

1. What parts are needed to make Wood table 40"?
2. Do we have enough parts to make 100 Deluxe wood table 60"?
3. What products require a table brace?
4. How much will the parts cost to make product Wood table 60"?

Using the Arrow tool add the relationships between the entities.

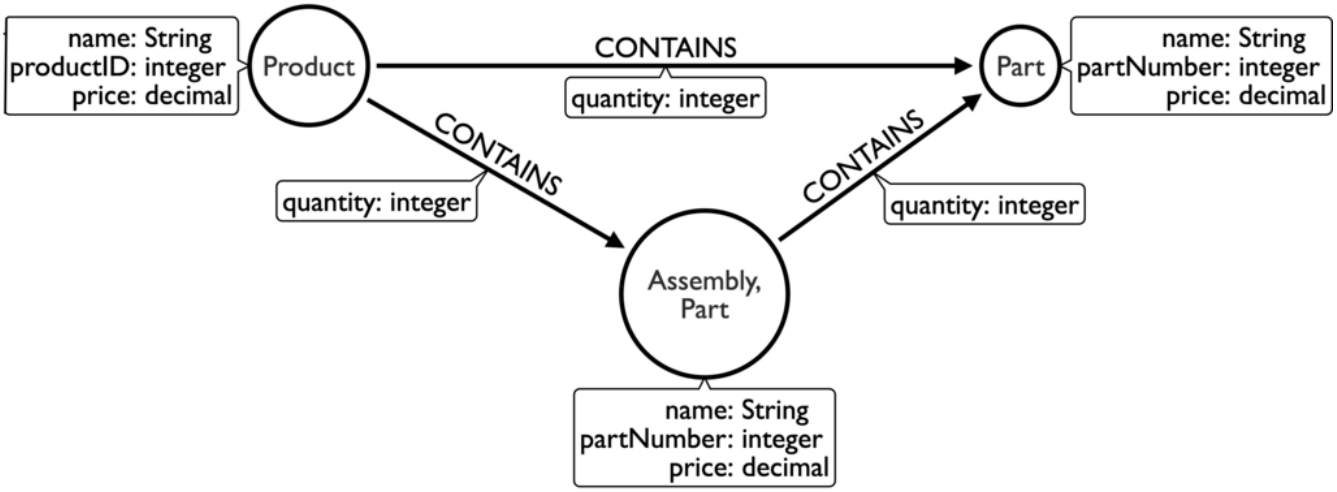
And here is the entity model:



Focus on type and directions first. Then check for any qualifiers that will help answer questions. Should these qualifiers be types or properties? Why?

Exercise 3 solution

Here is what your graph data model might look like with relationships added:



Here is the markup:

```
<ul class="graph-diagram-markup" data-internal-scale="2.01" data-external-
scale="1">
  <li class="node" data-node-id="0" data-x="283.8195928886757" data-y="-
6.306578837493484">
    <span class="caption">Part</span><dl
class="properties"><dt>name</dt><dd>String</dd><dt>partNumber</dt><dd>inte
ger</dd><dt>price</dt><dd>decimal</dd></dl></li>
  <li class="node" data-node-id="1" data-x="-333.52554559384123" data-y="-
6.306578837493484">
    <span class="caption">Product</span><dl
class="properties"><dt>name</dt><dd>String</dd><dt>productID</dt><dd>integ
er</dd><dt>price</dt><dd>decimal</dd></dl></li>
  <li class="node" data-node-id="2" data-x="9.793659703648462" data-
y="191.47083299670058">
    <span class="caption">Assembly, Part</span><dl
class="properties"><dt>name</dt><dd>String</dd><dt>partNumber</dt><dd>inte
ger</dd><dt>price</dt><dd>decimal</dd></dl></li>
  <li class="relationship" data-from="1" data-to="0">
    <span class="type">CONTAINS</span><dl
class="properties"><dt>quantity</dt><dd>integer</dd></dl></li>
  <li class="relationship" data-from="1" data-to="2">
    <span class="type">CONTAINS</span><dl
class="properties"><dt>quantity</dt><dd>integer</dd></dl></li>
  <li class="relationship" data-from="2" data-to="0">
    <span class="type">CONTAINS</span><dl
class="properties"><dt>quantity</dt><dd>integer</dd></dl></li>
</ul>
```

Again, your model need not exactly match this one. Remember, the first model is never perfect.

Example: Detailed sample data for the BOM application

Thus far, you have defined the entities and relationships for the BOM application. Just to fill in more details for the data model, here is more sample data that might be useful in furthering the design of the model. This particular sample data is useful, especially when discussing how well the design addresses the questions and how well the application can scale.

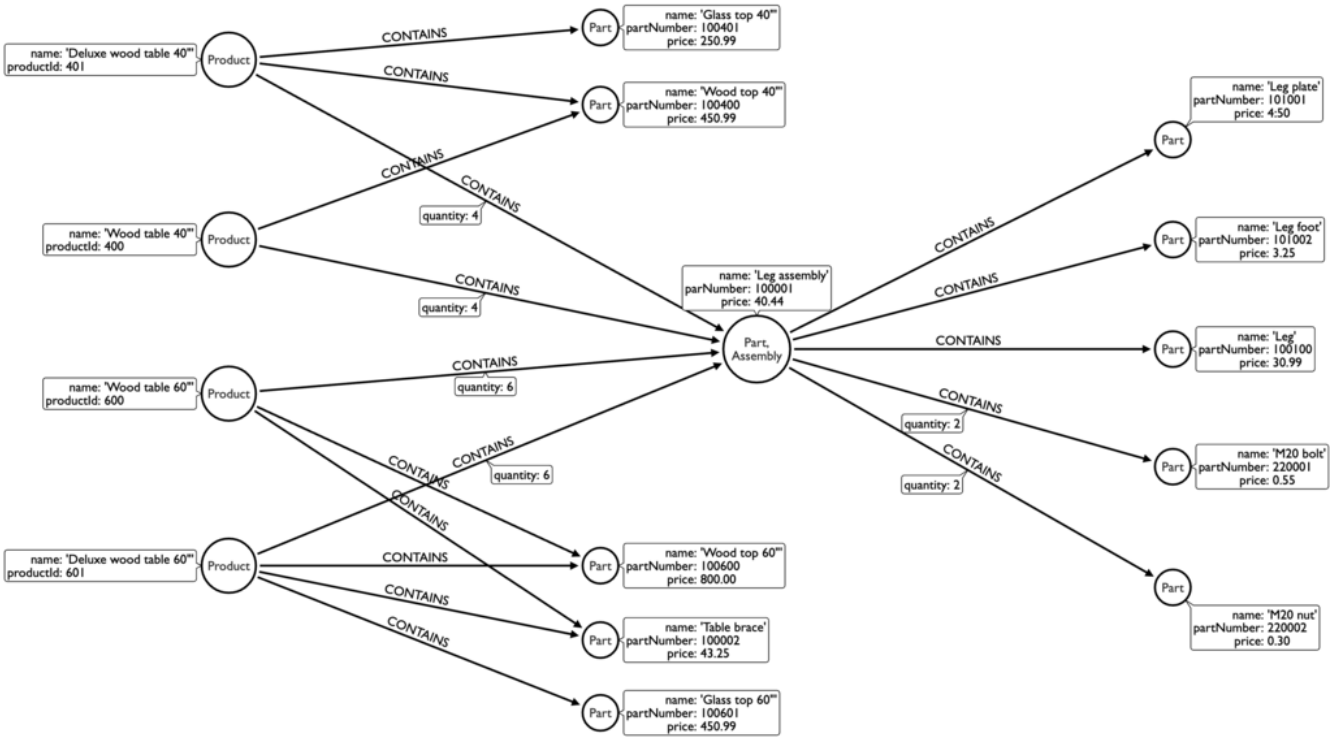
Here is the Arrow tool markup for the sample data for the BOM application:

```
<ul class="graph-diagram-markup" data-internal-scale="0.75" data-external-
scale="1">
  <li class="node" data-node-id="0" data-x="-345.4412714640299" data-
y="355.6422119140624">
    <span class="caption">Product</span><dl
class="properties"><dt>name</dt><dd>'Wood table
60"'</dd><dt>productId</dt><dd>600</dd></dl></li>
  <li class="node" data-node-id="1" data-x="-345.4412714640299" data-
y="17.62464618682847">
    <span class="caption">Product</span><dl
class="properties"><dt>name</dt><dd>'Deluxe wood table
40"'</dd><dt>productId</dt><dd>401</dd></dl></li>
  <li class="node" data-node-id="2" data-x="-345.4412714640299" data-
y="199.74443022410088">
    <span class="caption">Product</span><dl
class="properties"><dt>name</dt><dd>'Wood table
40"'</dd><dt>productId</dt><dd>400</dd></dl></li>
  <li class="node" data-node-id="3" data-x="-345.4412714640299" data-
y="529.8155937194823">
    <span class="caption">Product</span><dl
class="properties"><dt>name</dt><dd>'Deluxe wood table
60"'</dd><dt>productId</dt><dd>601</dd></dl></li>
  <li class="node" data-node-id="4" data-x="30.527327269315723" data-
y="63.11414146423326">
    <span class="caption">Part</span><dl
class="properties"><dt>name</dt><dd>'Wood top
40"'</dd><dt>partNumber</dt><dd>100400</dd><dt>price</dt><dd>450.99</dd></
dl></li>
  <li class="node" data-node-id="5" data-x="609.4605747858683" data-
y="551.8277514775586">
    <span class="caption">Part</span><dl
class="properties"><dt>name</dt><dd>'M20
nut'</dd><dt>partNumber</dt><dd>220002</dd><dt>price</dt><dd>0.30</dd></dl
></li>
  <li class="node" data-node-id="6" data-x="30.527327269315723" data-
y="605.0613317489621">
    <span class="caption">Part</span><dl
class="properties"><dt>name</dt><dd>'Table
brace'</dd><dt>partNumber</dt><dd>100002</dd><dt>price</dt><dd>43.25</dd><
/dl></li>
  <li class="node" data-node-id="7" data-x="188.62522285183277" data-
y="310.58522319793684">
    <span class="caption">Part, Assembly</span><dl
```

```
class="properties"><dt>name</dt><dd>'Leg
assembly'</dd><dt>parNumber</dt><dd>100001</dd><dt>price</dt><dd>40.44</dd
></dl></li>
  <li class="node" data-node-id="8" data-x="609.4605747858683" data-
y="429.5683333079018">
    <span class="caption">Part</span><dl
class="properties"><dt>name</dt><dd>'M20
bolt'</dd><dt>partNumber</dt><dd>220001</dd><dt>price</dt><dd>0.55</dd></d
l></li>
  <li class="node" data-node-id="9" data-x="609.4605747858683" data-
y="199.74443022410088">
    <span class="caption">Part</span><dl
class="properties"><dt>name</dt><dd>'Leg
foot'</dd><dt>partNumber</dt><dd>101002</dd><dt>price</dt><dd>3.25</dd></d
l></li>
  <li class="node" data-node-id="10" data-x="609.4605747858683" data-
y="98.58790914217629">
    <span class="caption">Part</span><dl
class="properties"><dt>name</dt><dd>'Leg
plate'</dd><dt>partNumber</dt><dd>101001</dd><dt>price</dt><dd>4:50</dd></
dl></li>
  <li class="node" data-node-id="11" data-x="609.4605747858683" data-
y="310.58522319793684">
    <span class="caption">Part</span><dl
class="properties"><dt>name</dt><dd>'Leg'</dd><dt>partNumber</dt><dd>10010
0</dd><dt>price</dt><dd>30.99</dd></dl></li>
  <li class="node" data-node-id="12" data-x="30.527327269315723" data-
y="678.7611147562659">
    <span class="caption">Part</span><dl
class="properties"><dt>name</dt><dd>'Glass top
60"'</dd><dt>partNumber</dt><dd>100601</dd><dt>price</dt><dd>450.99</dd></
dl></li>
  <li class="node" data-node-id="13" data-x="30.527327269315723" data-y="-
14.911466916402228">
    <span class="caption">Part</span><dl
class="properties"><dt>name</dt><dd>'Glass top
40"'</dd><dt>partNumber</dt><dd>100401</dd><dt>price</dt><dd>250.99</dd></
dl></li>
  <li class="node" data-node-id="14" data-x="30.527327269315723" data-
y="529.8155937194823">
    <span class="caption">Part</span><dl
class="properties"><dt>name</dt><dd>'Wood top
60"'</dd><dt>partNumber</dt><dd>100600</dd><dt>price</dt><dd>800.00</dd></
dl></li>
  <li class="relationship" data-from="2" data-to="4">
    <span class="type">CONTAINS</span>
  </li>
  <li class="relationship" data-from="1" data-to="4">
    <span class="type">CONTAINS</span>
  </li>
  <li class="relationship" data-from="1" data-to="13">
    <span class="type">CONTAINS</span>
  </li>
  <li class="relationship" data-from="1" data-to="7">
    <span class="type">CONTAINS</span><dl
class="properties"><dt>quantity</dt><dd>4</dd></dl></li>
```

```
  <li class="relationship" data-from="2" data-to="7">
    <span class="type">CONTAINS</span><dl
class="properties"><dt>quantity</dt><dd>4</dd></dl></li>
  <li class="relationship" data-from="0" data-to="7">
    <span class="type">CONTAINS</span><dl
class="properties"><dt>quantity</dt><dd>6</dd></dl></li>
  <li class="relationship" data-from="0" data-to="14">
    <span class="type">CONTAINS</span>
  </li>
  <li class="relationship" data-from="3" data-to="12">
    <span class="type">CONTAINS</span>
  </li>
  <li class="relationship" data-from="3" data-to="14">
    <span class="type">CONTAINS</span>
  </li>
  <li class="relationship" data-from="3" data-to="6">
    <span class="type">CONTAINS</span>
  </li>
  <li class="relationship" data-from="0" data-to="6">
    <span class="type">CONTAINS</span>
  </li>
  <li class="relationship" data-from="3" data-to="7">
    <span class="type">CONTAINS</span><dl
class="properties"><dt>quantity</dt><dd>6</dd></dl></li>
  <li class="relationship" data-from="7" data-to="11">
    <span class="type">CONTAINS</span>
  </li>
  <li class="relationship" data-from="7" data-to="10">
    <span class="type">CONTAINS</span>
  </li>
  <li class="relationship" data-from="7" data-to="9">
    <span class="type">CONTAINS</span>
  </li>
  <li class="relationship" data-from="7" data-to="8">
    <span class="type">CONTAINS</span><dl
class="properties"><dt>quantity</dt><dd>2</dd></dl></li>
  <li class="relationship" data-from="7" data-to="5">
    <span class="type">CONTAINS</span><dl
class="properties"><dt>quantity</dt><dd>2</dd></dl></li>
</ul>
```

Here is what the sample data might look like in the UI of the Arrow tool:

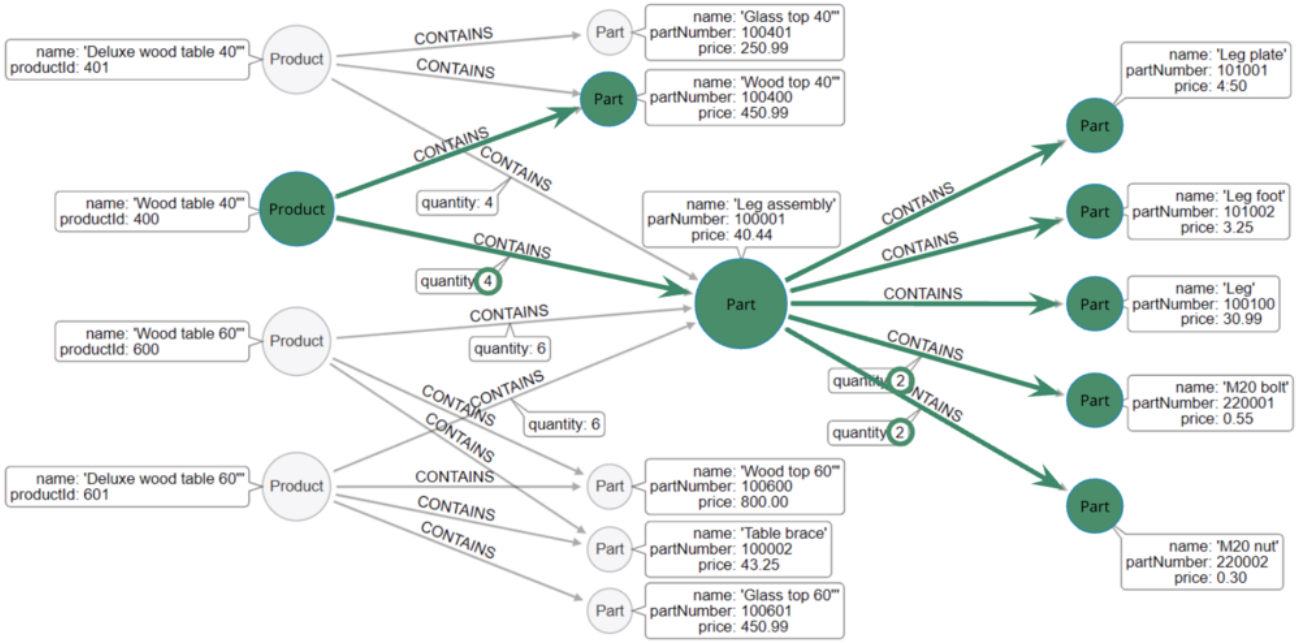


Step 6: Test the questions against the model

1. Understand the domain.
2. Create high-level sample data.
3. Define specific questions for the application.
4. Identify entities.
5. Identify connections between entities.
6. **Test the questions against the model.**
7. Test scalability.

Testing the model - 1

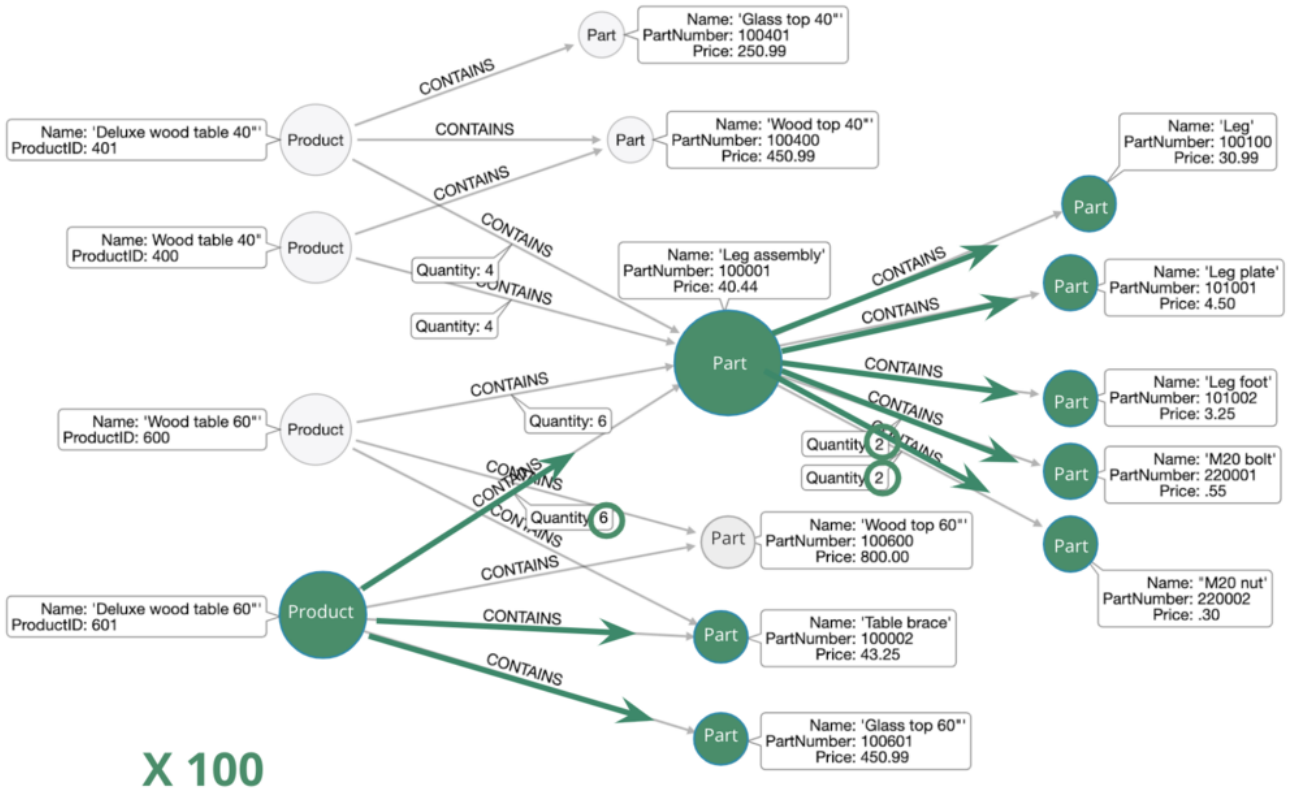
What parts are needed to make Wood table 40"?



To answer the first question, we would anchor on the Wood table 40" node, and iteratively traverse all CONTAINS until we get all of the parts required to make the product. Multiply by the quantity property as you go.

Testing the model - 2

Do we have enough parts to make 100x Deluxe wood table 60"?



The application code that supports this question would need to come up with the inventory of parts needed to assemble the Deluxe wood table 60" and then multiply the parts list by 100. The application would then be integrated with an inventory application to truly answer the question.

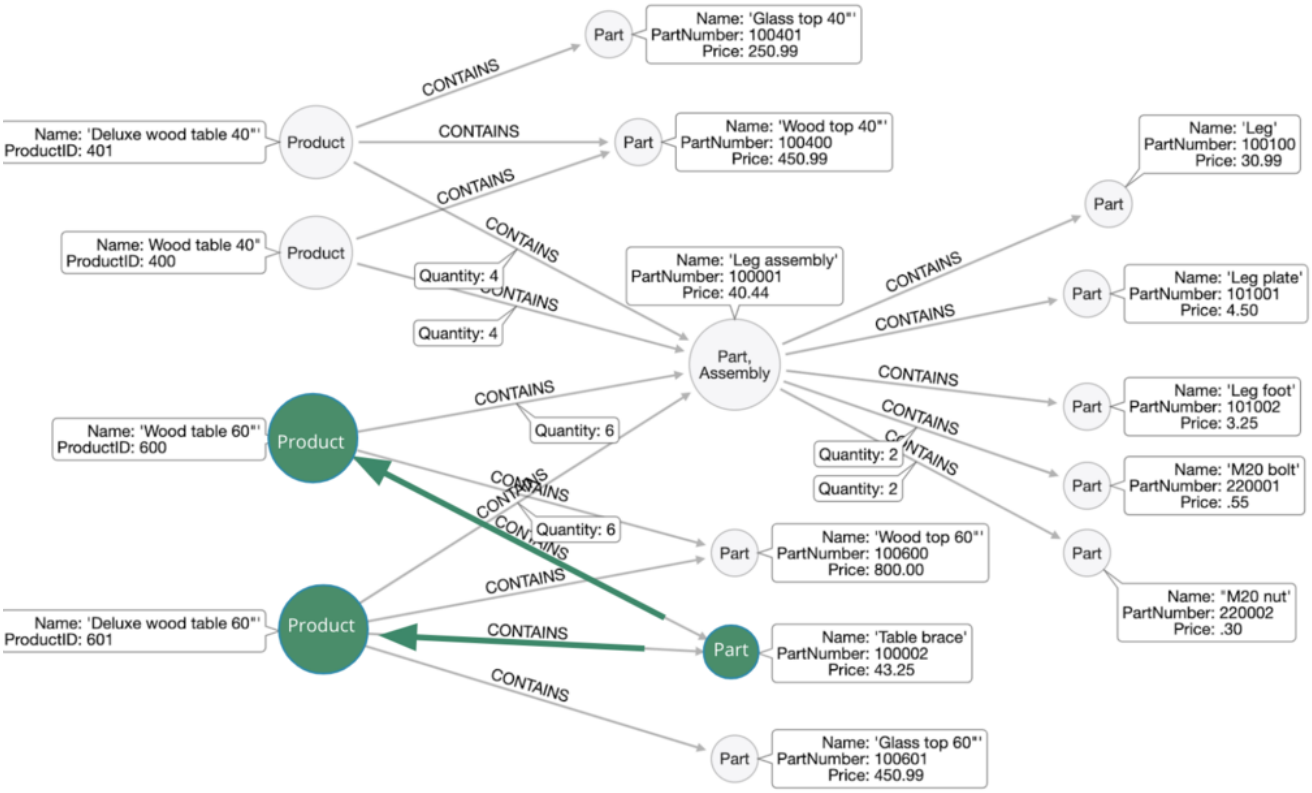
Frequently, during the entity identification stage, the question of inventory comes up. Different people will advocate for or against it. Both cases work, but with different strengths and weaknesses.

If inventory is in the graph, the application architecture can be simpler—no need to integrate two data sources at query time. The downside is that stock numbers change frequently, and whenever they do, you will need to update the graph data. Neo4j is not specifically bad for frequent updates, but it offers no benefits in that area relative to other data management solutions.

Ideologically, we at Neo4j see graph databases as a problem-solving complement to the existing data management paradigm. If you were maintaining stock data somewhere else anyway, there is no inherent need to move it into Neo4j.

Testing the model - 3

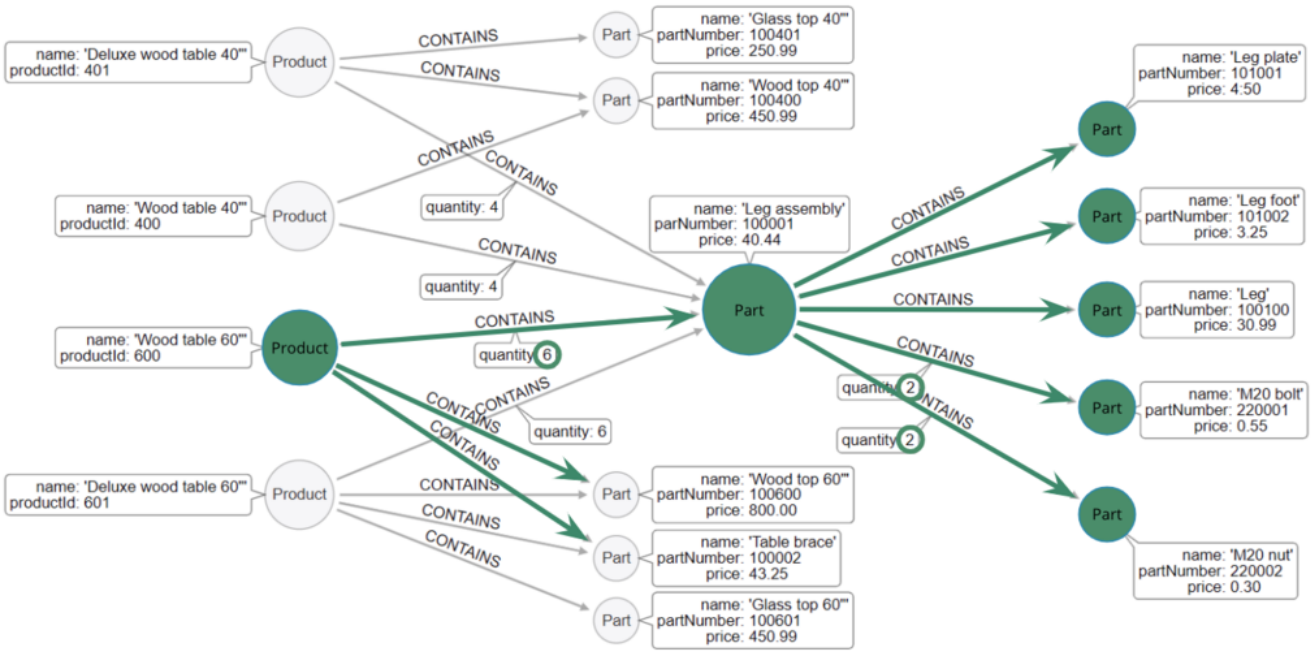
What products require a table brace?



For this question, the application would anchor on the Part with the name table brace. Then it would traverse backwards along each CONTAINS to find the products that use it.

Testing the model - 4

How much will the parts cost to make Wood table 60"?



We anchor on the Wood table 60" product, and iteratively traverse each CONTAINS to the parts that the product contains. The application would simply add up the prices of the parts (multiplied by quantity when applicable) to come up with the total price.

During the entity identification stage, there is often a debate about whether Assembly nodes should contain a price, or if price should be kept only on the Part leaf nodes. That debate may even extend to whether price should be kept on Product nodes. Again, there is no “right” answer, only tradeoffs to be aware of.

If price is only on Parts, then this query needs to traverse further in order to find the answer. The benefit is that data updates are easier. If Assembly has a price, then that price is affected by any price changes of the sub-parts; ditto if price is on Products. The added complexity in data maintenance is generally undesirable. However, one big benefit of putting price at the Assembly level is that the Cypher for this question is simpler and more predictable. You can guarantee that Price is available exactly one hop away from the anchor node in all cases. This modeler felt that that benefit balanced out the added data maintenance expense.

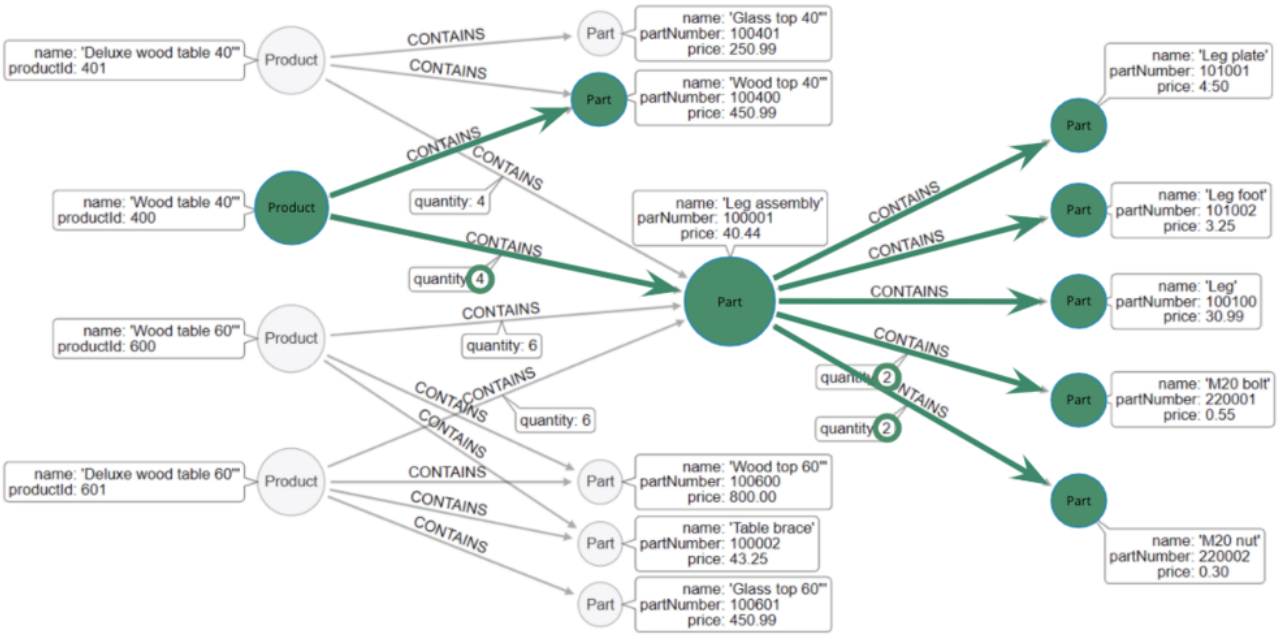
Step 7: Test scalability

1. Understand the domain.
2. Create high-level sample data.
3. Define specific questions for the application.
4. Identify entities.
5. Identify connections between entities.
6. Test the questions against the model.
7. **Test scalability.**

Testing scalability

Here are some questions you should answer:

- How many products?
- How many parts?
- How often are products added?
- How often do prices change?
- Are prices based upon time?
- Is inventory part of the model?



Scalability questions should be asked by stakeholders and developers. Developers will have a better idea of the challenges with implementing the model as it scales so they are a key player in doing the scalability analysis. Stakeholders would have a better idea of how many products and parts, how often prices change etc.

Here, we are only looking for conceptual scalability concerns. We also do this at the level of individual query performance during the implemtation of the graph data model.

Check your understanding

Question 1

What component of a graph data model is used to model the nouns of the questions for the domain?

Select the correct answer.

- ☐ Relationship
- ☐ Property

- [x] Entity
- [ ] Data source

Question 2

What component of a graph data model is used to model the verbs of the questions for the domain?

Select the correct answer.

- [x] Relationship
- [ ] Property
- [ ] Entity
- [ ] Behavior

Question 3

When you create the model for your application, at a minimum, what must you specify for a relationship?

Select the correct answers.

- [x] Type
- [ ] Weight
- [x] Direction
- [x] From and to entities

Summary

You should now be able to:

- Describe the domain for a model.
- Define the questions for the domain.
- Identify entities from the questions for the domain.
- Use the Arrows Tool to model the domain.
- Identify the connections between entities.
- Describe how to test the initial model.

Graph Data Modeling Core Principles

About this module

At the end of this module, you should be able to:

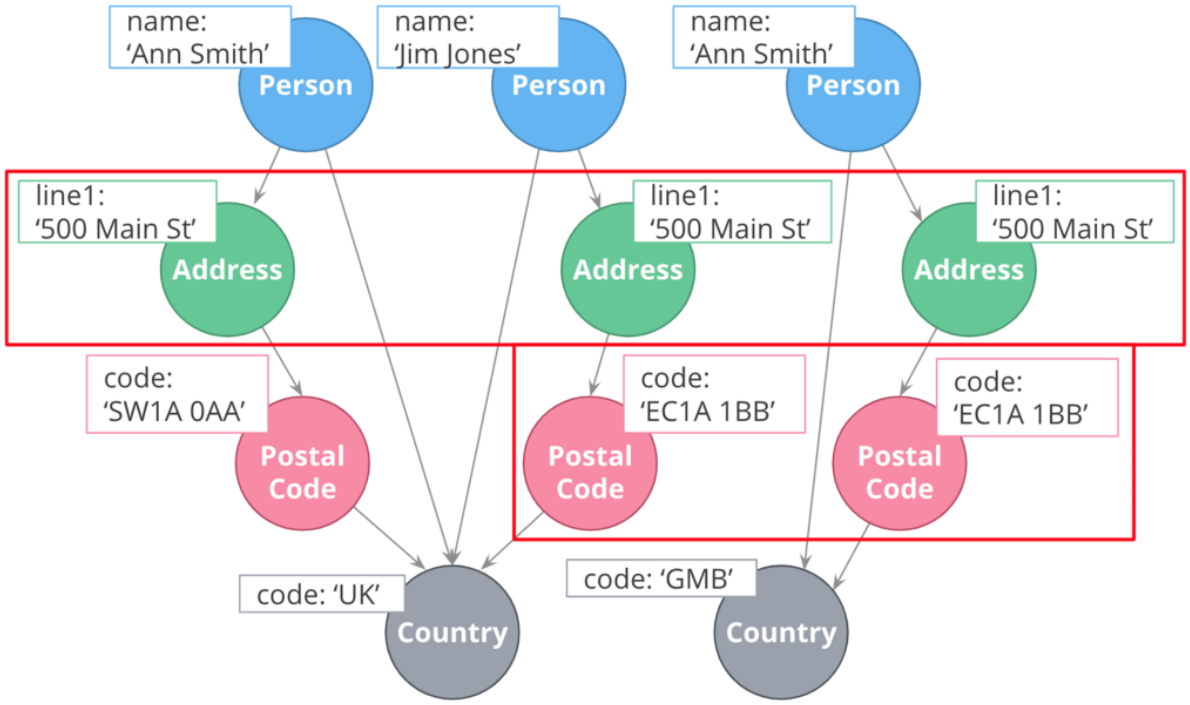
- Describe graph data modeling best practices for modeling:
  - Nodes (entities)
  - Relationships
  - Properties
- Describe data accessibility

Best practices for modeling nodes

Your model should address:

- Uniqueness of nodes
- Complex data

Uniqueness of nodes: before



NOTE

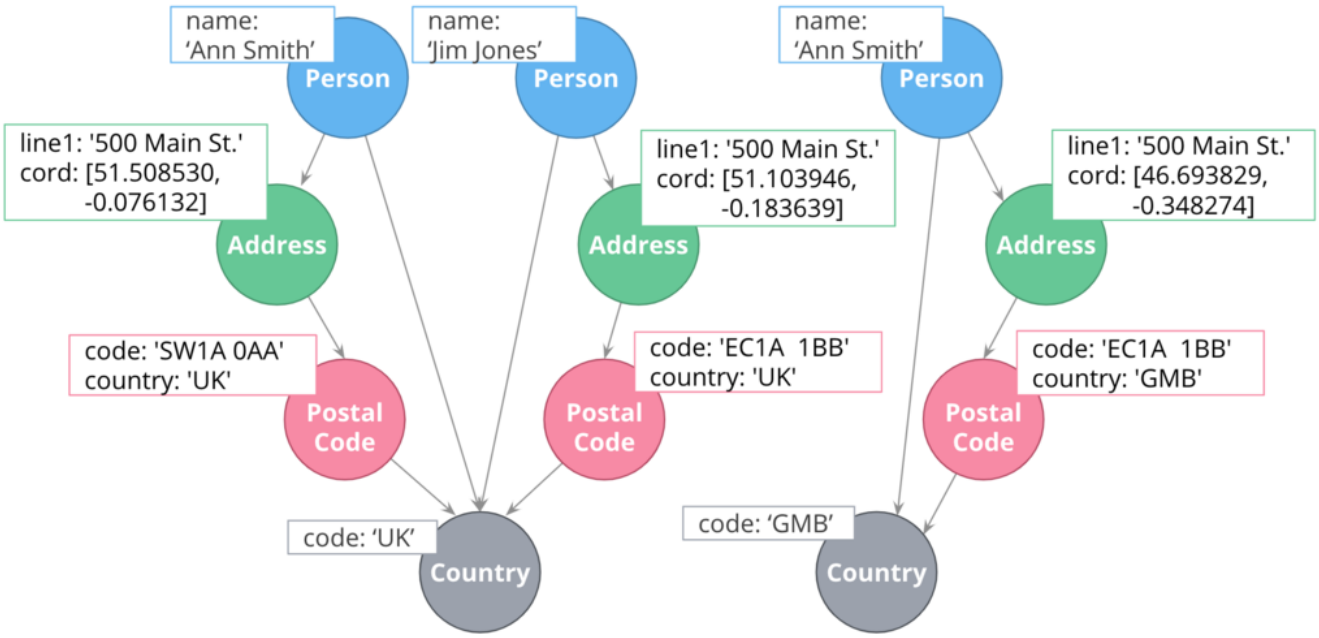
The Country nodes are considered **super nodes** (a node with lots of fan-in or fan-out). You need to be careful about using them in your design and developers in particular need to be mindful of queries that might select all paths in or out of the super node.

In models like this one, an address should be uniquely identified because it should represent a

single point of data in the graph. This is important in fraud detection applications where variations of the same address may be used to make a fraudulent transaction against the application.

In this particular example, it is clear from the wider context that the various 500 Main St nodes are all distinct places. We also see that postal code is not unique and we want to provide uniqueness to it by adding the country.

Uniqueness of nodes: after

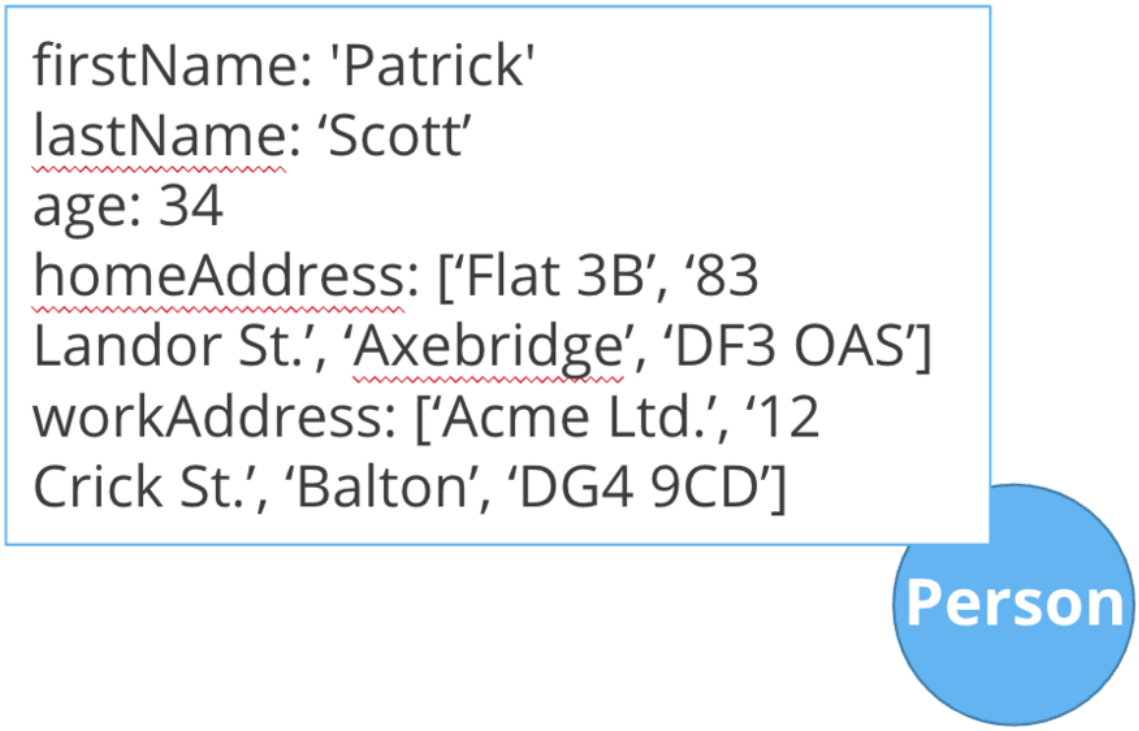


Here is a solution where we have ensured that the Address and PostalCode nodes are unique.

We consider it a best practice to always have a property (or set of properties) that uniquely identify a node. Here, we have added a geolocation property to do so. The geolocation property will likely never be used in a query, but it can be used to differentiate nodes.

There is a trade-off, however as adding uniqueness to a node such Address as it is harder to anchor just on the Line1 value, and may be more difficult to modify the graph data later.

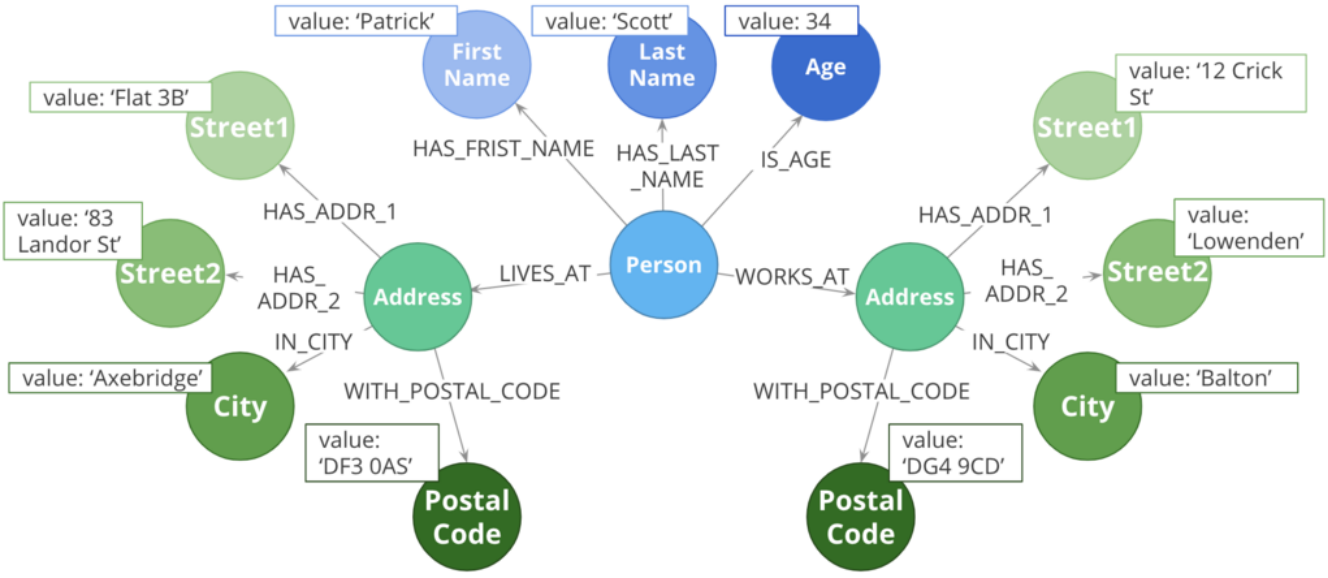
Complex data



You need to strike a balance between number of properties that represent complex data vs. multiple nodes and relationships.

Here we have a property node that contains properties that contain complex data.

Use fanout judiciously for complex data



- Reduce property duplication.
- Reduce gather-and-inspect.

Here we show an extreme implementation of fanout. For modeling complex data, the previous example with all properties in a node and this example where each property is in its own node are usually suboptimal.

In general, you use fanout to do one of two things.

1. Reduce duplication of properties. Instead of having a repeated property on every node, you can instead have all of those nodes connected to a shared node with that property. This can make data updates massively easier.
2. Reduce gather-and-inspect behavior during a traversal. In the one node example, if we want to find every address in the city of Axebridge, we would need to check the properties on **every** Person node, then discard most of them. This is grossly inefficient. In this multiple node case, this is a simple matter of locating the singular Axebridge node, and traversing to every Address node connected to it. This model has no "wasted" hops.

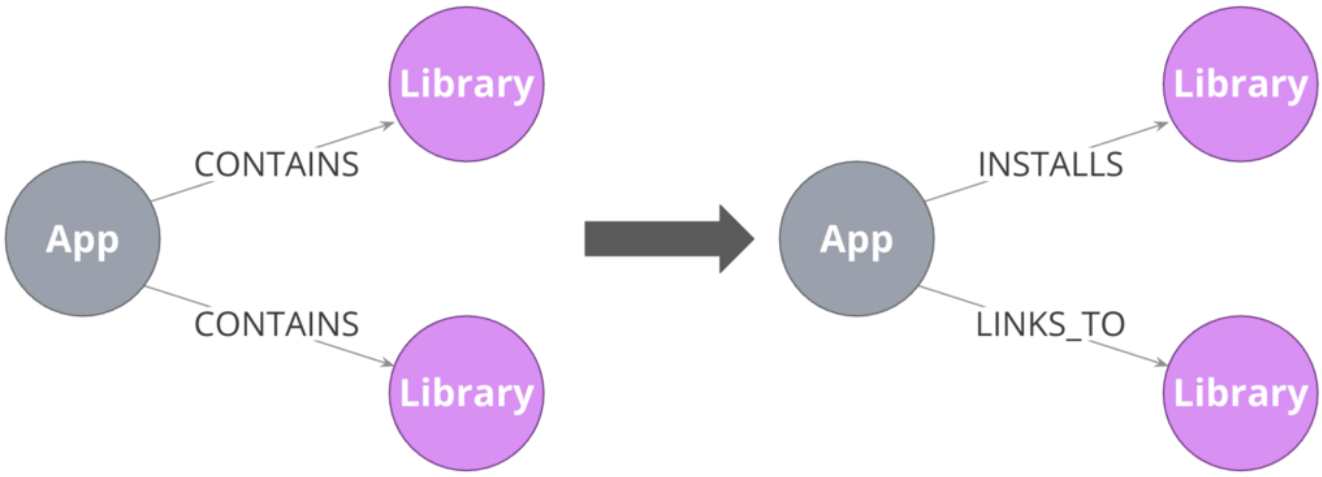
As a result, you generally use fanout for anchors and traversing the graph. You almost never see fanout used for output, unique identifier, or decorator properties, because it makes traversal a few hops longer for no real benefit. This is why this maximum-fanout case is usually undesirable. It is almost never the case that **every** property is an anchor or used for the traversal!

Best practices for modeling relationships

Your model should address:

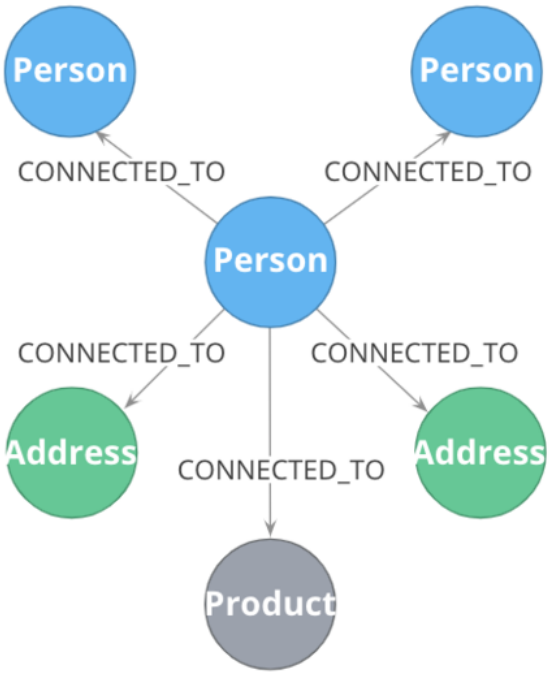
- Using specific relationship types.
- Reducing symmetric relationships.
- Using types vs. properties.

Using specific relationship types

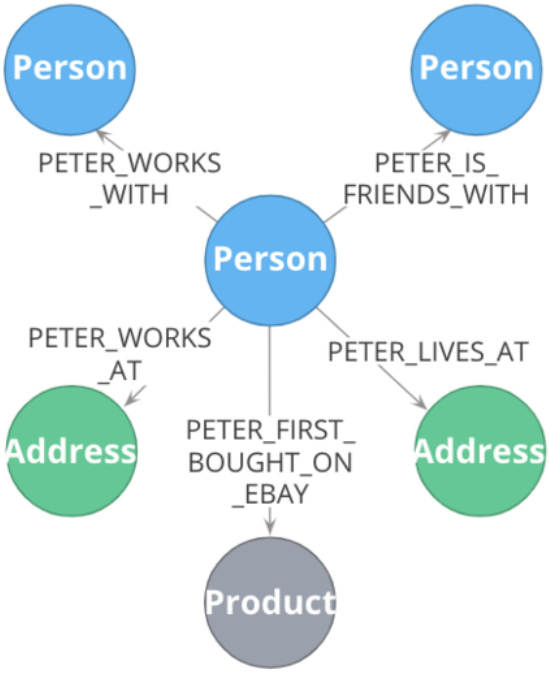


Being specific with property types allows you to reduce gather-and-inspect behavior. In this case, if you are only interested in what libraries will be INSTALLED by an app, the specific types on the right saves you some wasted traversal.

But… not too specific



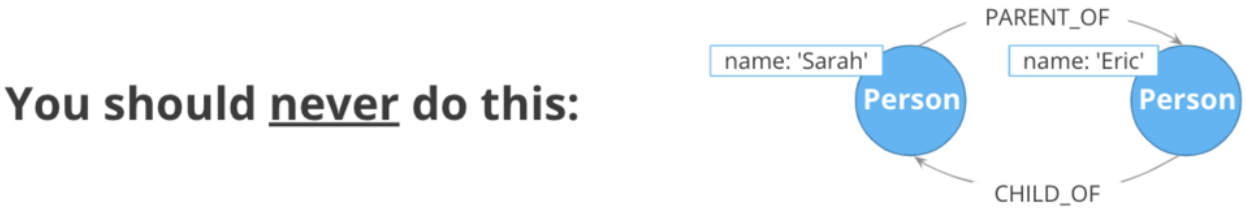
Not specific enough

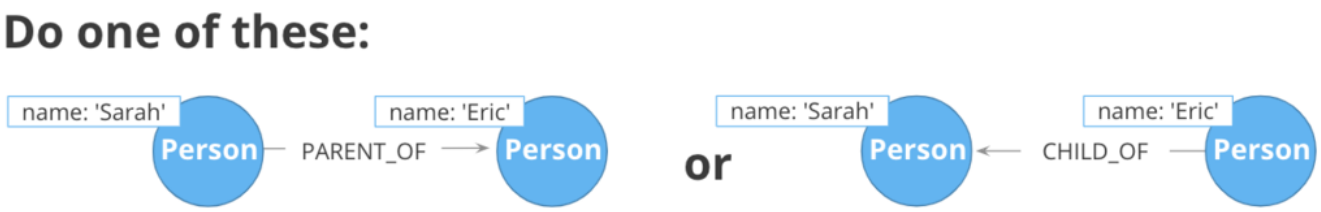


Too specific

But it is possible to be **too** specific! The model on the right makes it impossible to write generalized queries. If you want to find every person who works at a given address, you would need to write a massive WHERE clause (using OR) to include every person's name in the relationship type

No symmetric relationships





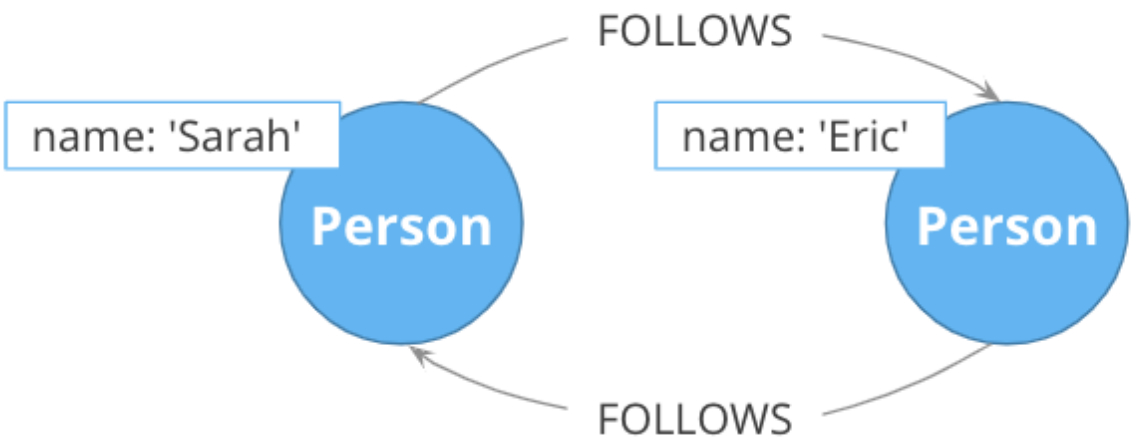
Semantically symmetric relationships present two problems.

First, they are a form of needless data duplication. PARENT_OF and CHILD_OF mean exactly the same thing. You cannot have one be true and the other not.

Second, they allow you to violate the Cypher expectation of relationship uniqueness. Semantically, you have two identical relationships—they just look different technically. This allows you to traverse the **same** relationship twice.

Relationships require space in the graph so minimizing their numbers is always a good thing.

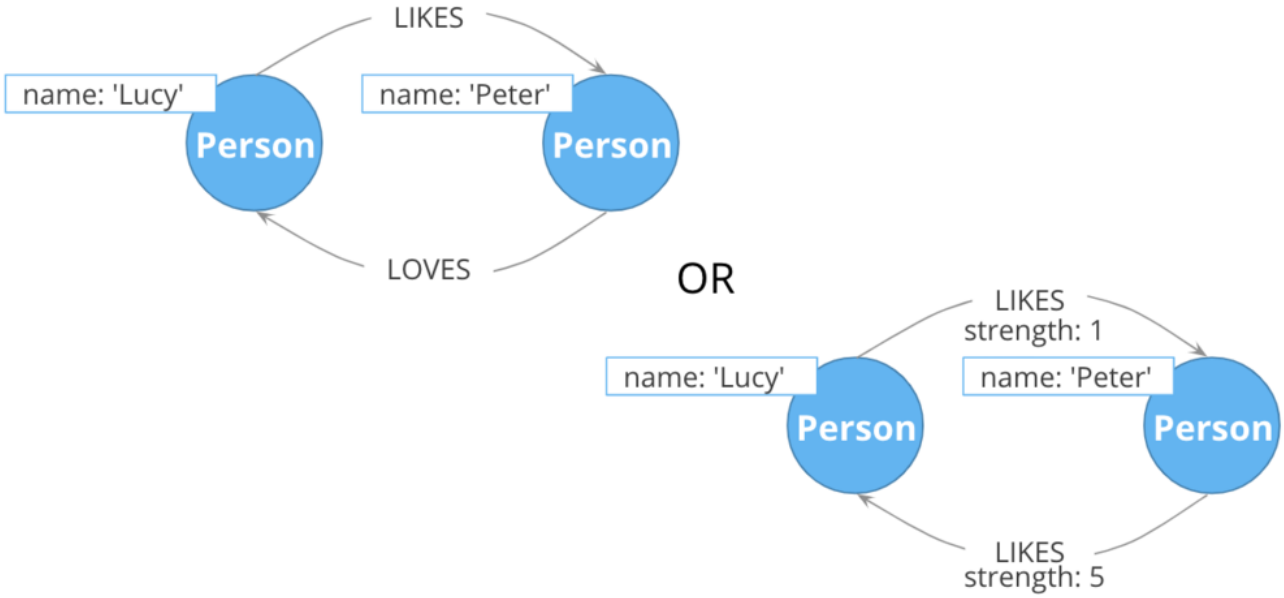
Semantics of symmetry are important



Not all mutual relationships are semantically symmetric.

Here is an example where the direction of the FOLLOWS relationship on Twitter is significant. It matters who has followed who.

Using types vs. properties

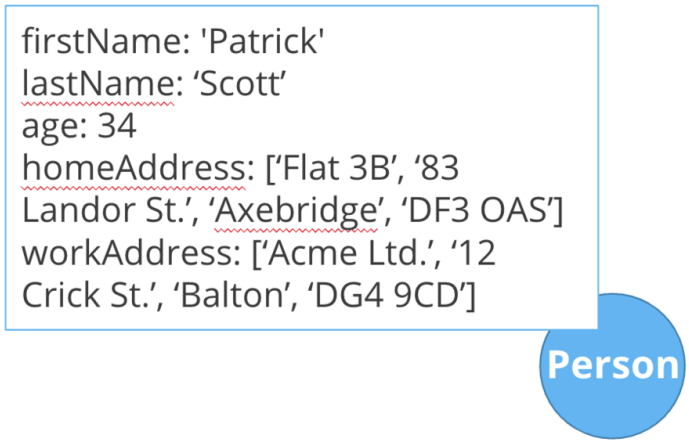


Both of these models represent the same idea in different ways. Neither is strictly superior; both are optimized for a certain kind of question.

For example, if you want to find all LOVES relationships, but ignore the weaker LIKES ones, the top model is best. Traversal will not involve any gather-and-inspect. On the other hand, if you want to rank the strength of all relationships, the below model is better. The Cypher string required is much simpler, and we know there will not be any gather-and-inspect discards because we want everything anyway.

Property best practices

- Property lookups have a cost.
- Parsing a complex property adds more cost.



- Anchors and properties used for traversal should be as simple as possible.
- Identifiers, outputs, and decoration are OK as complex values.

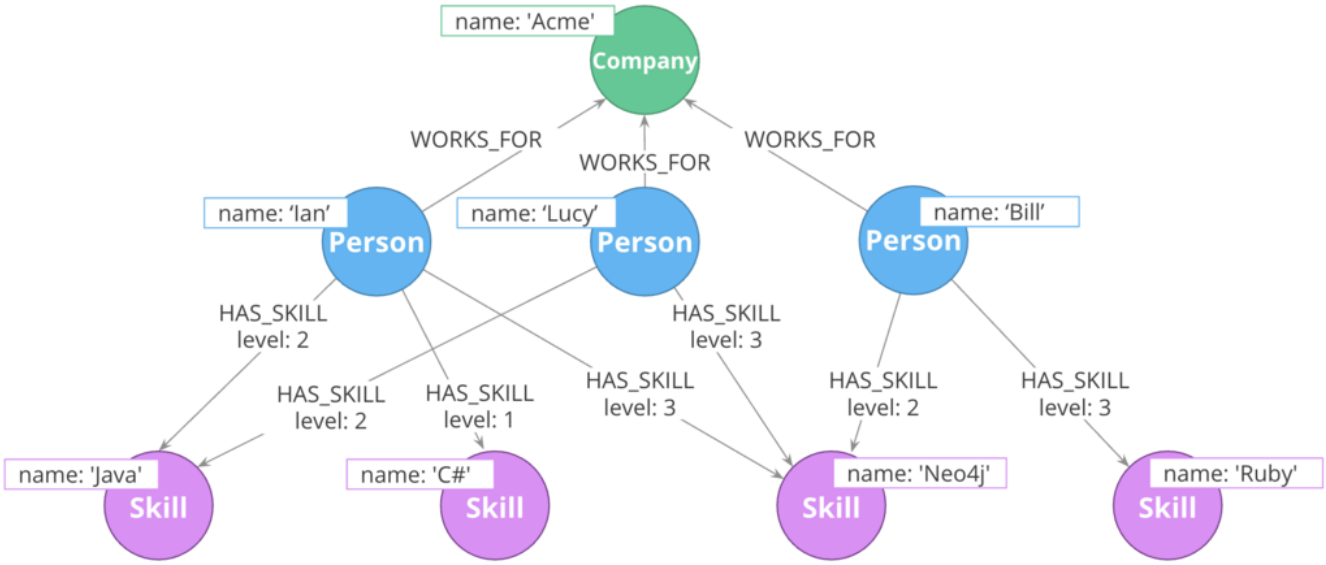
When talking about properties, every best practice has exceptions. In the case of property value complexity, it depends on how the property is used.

Anchors and traversal paths that use property values need to be parsed at query time. If the cost of parsing them is high, querying slows down. Other properties, though, just need to be returned as-is because they do not require parsing. Complexity there does not matter much.

In other words, this "bad model" from before might actually be OK. If those addresses are never used as an anchor or for determining how to traverse the graph, and only as an output, then their complexity will not be an issue.

Data accessibility

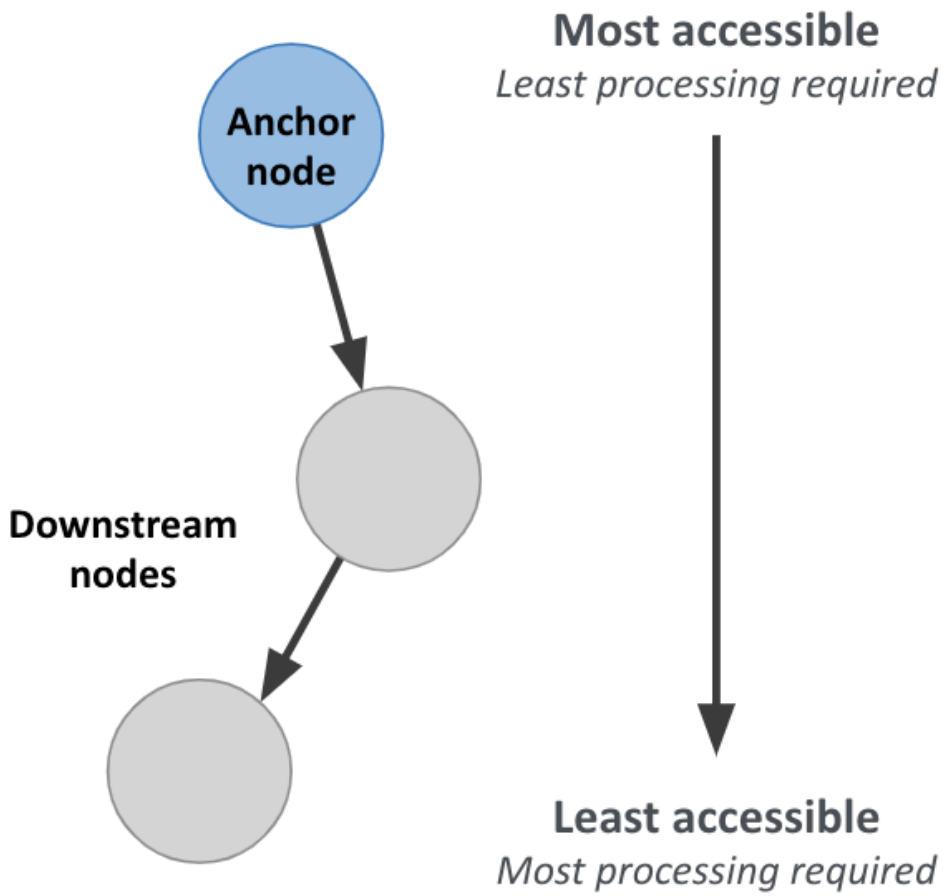
For each query, how much work must Neo4j do to evaluate if the traversal represents a "good" path or a "bad" one?



Neo4j allows you to put data in labels, properties, and relationship types. In most cases, there is a way you can model your graph so that a given data item is in any one of these three locations.

When designing or refactoring a model, you can estimate performance by checking how "accessible" the data items are that represent the important queries of your application.

Hierarchy of accessibility



For each data object, how much work must Neo4j do to evaluate if this is a "good" path or a "bad"

one?

1. Anchor node label, indexed anchor node properties
2. Relationship types
3. Non-indexed anchor node properties
4. Downstream node labels
5. Relationship properties, downstream node properties

Anchor node data which could be a node label or a node property value is cheapest to access—it involves no traversal at all, and thus no wasted traversal. Indexed anchor properties are much more accessible than non-indexed anchor properties.

Neo4j is designed for easy traversal, so relationship types are also very cheap to access. In fact, we recommend using types in Cypher queries always, because the cost of the type lookup is minimal. And if you do not use a type name in your traversal path, you will likely use something even less accessible, like downstream properties.

Downstream information such as labels and properties further down the traversal path are the most expensive thing to access. The further downstream, the more expensive the are. That is not to say they are necessarily deal-breakers, but elevating downstream data via a model change is one of the most reliable ways to improve query performance. For example, reduce how far downstream a query traversal must go to get the necessary data, or change the model so that the downstream data is available in a relationship type instead of a node label or property.

Keep in mind, however, that query performance is not the **only** metric that matters! Query simplicity, write/update speed, and the human-intuitiveness of a model are also important factors. When considering whether to elevate some data along this hierarchy, speed must be weighed against the impact this will have on those other factors.

Check your understanding

Question 1

What are some benefits of using fanout for your nodes?

Select the correct answers.

- ☐ Reduces the number of nodes in the graph.
- ☑ Reduces duplication of property values.
- ☐ Reduces the number of relationships defined in the graph.
- ☑ Reduces gather-and-inspect traversals during a query.

Question 2

Why is naming relationship types to be as specific as possible a benefit?

Select the correct answers.

- ☐ Reduces the number of relationships in the graph.
- ☑ Reduces traversals through nodes that are not necessary for the query.
- ☑ Reduces gather-and-inspect traversals during a query.
- ☐ Reduces the number of nodes in the graph.

Question 3

Which data should be most accessible for your queries?

Select the correct answers.

- ☑ Anchor node label
- ☑ Anchor node property that has an index
- ☐ Node property downstream that has an index
- ☐ Relationship properties

Summary

You should now be able to:

- Describe graph data modeling best practices for modeling:
  - Nodes (entities)
  - Relationships
  - Properties
- Describe data accessibility

Common Graph Structures

About this module

At the end of this module, you should be able to:

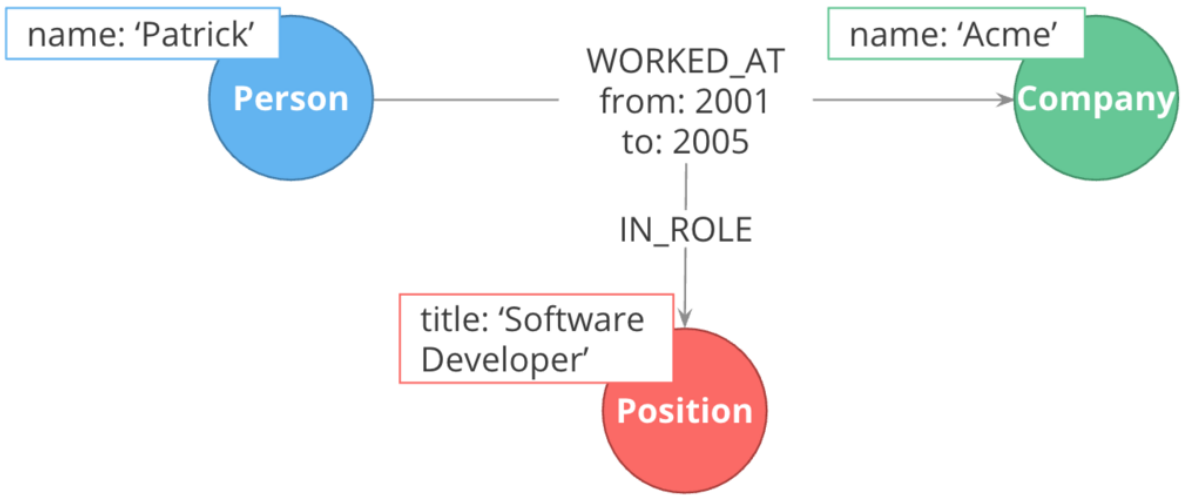
- Describe common graph structures used in modeling:
  - Intermediate nodes
  - Linked lists
  - Timeline trees
  - Multiple structures in a single graph

Intermediate nodes

You sometimes find cases where you need to connect more data to a relationship than can be fully captured in the properties. In other words, you want a relationship that connects more than two nodes. Mathematics allows this, with the concept of a hyperedge. This is impossible in Neo4j

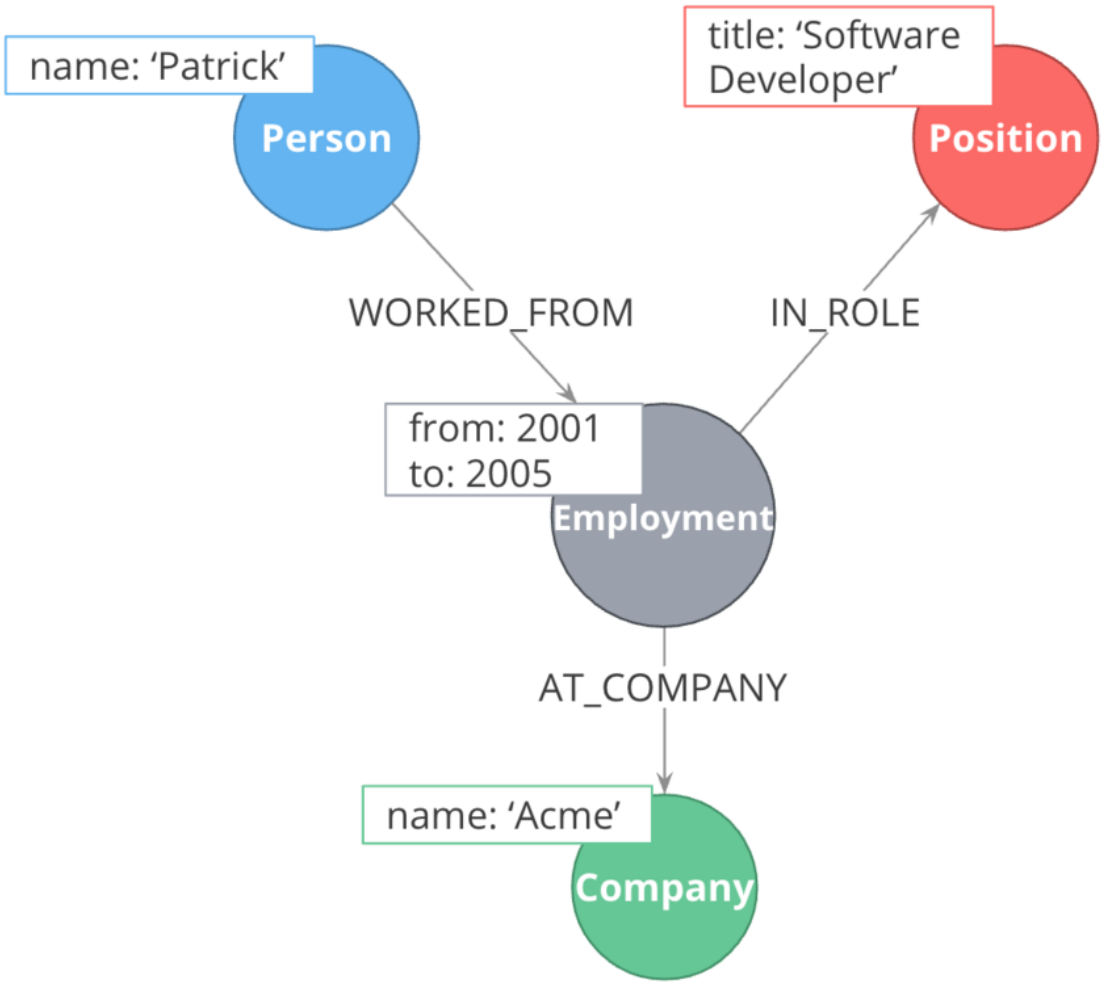
You create intermediate nodes when you need to:

- Connect more than two nodes in a single context.
  - Hyperedges (n-ary relationships)
- Relate something to a relationship.



Here we have the *WORKED_AT* relationship that has the from and to properties, but we need to associate the role with this period of work. In Neo4j, there is no way to create a relationship that connects a relationship to a node. Relationships can only connect nodes.

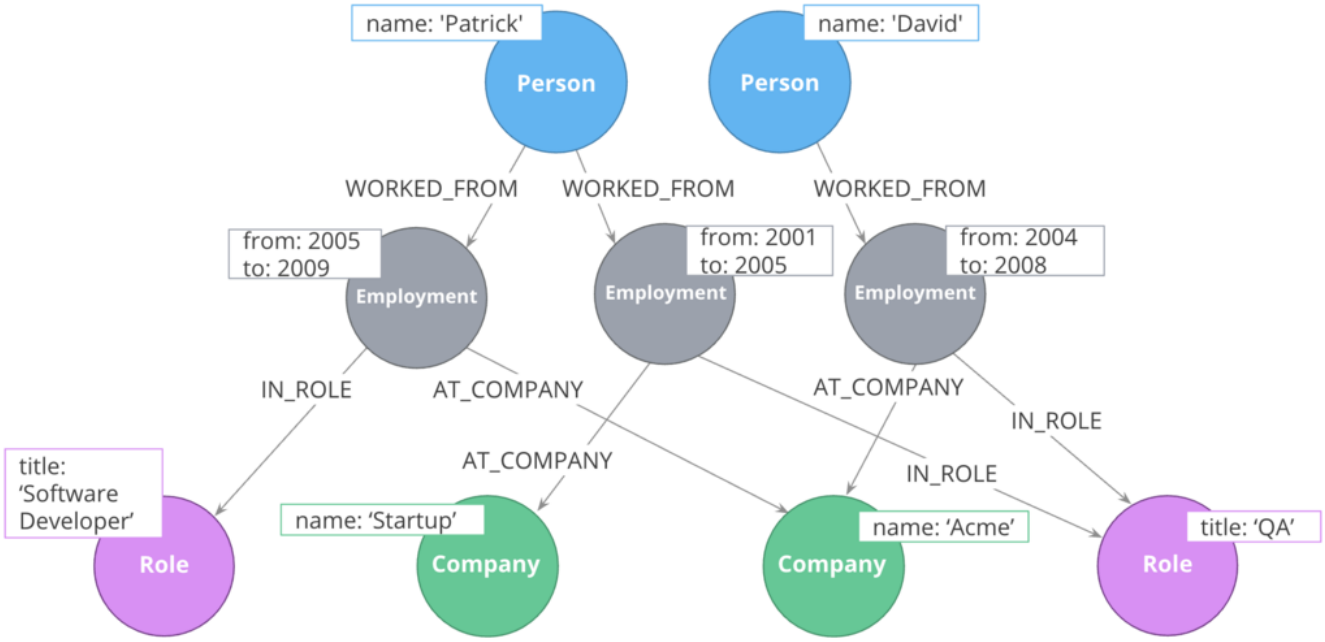
Using intermediate nodes



The solution is to add a connection point in the middle of the relationship. Since nodes are connection points, you simply create a node in the middle of the relationship.

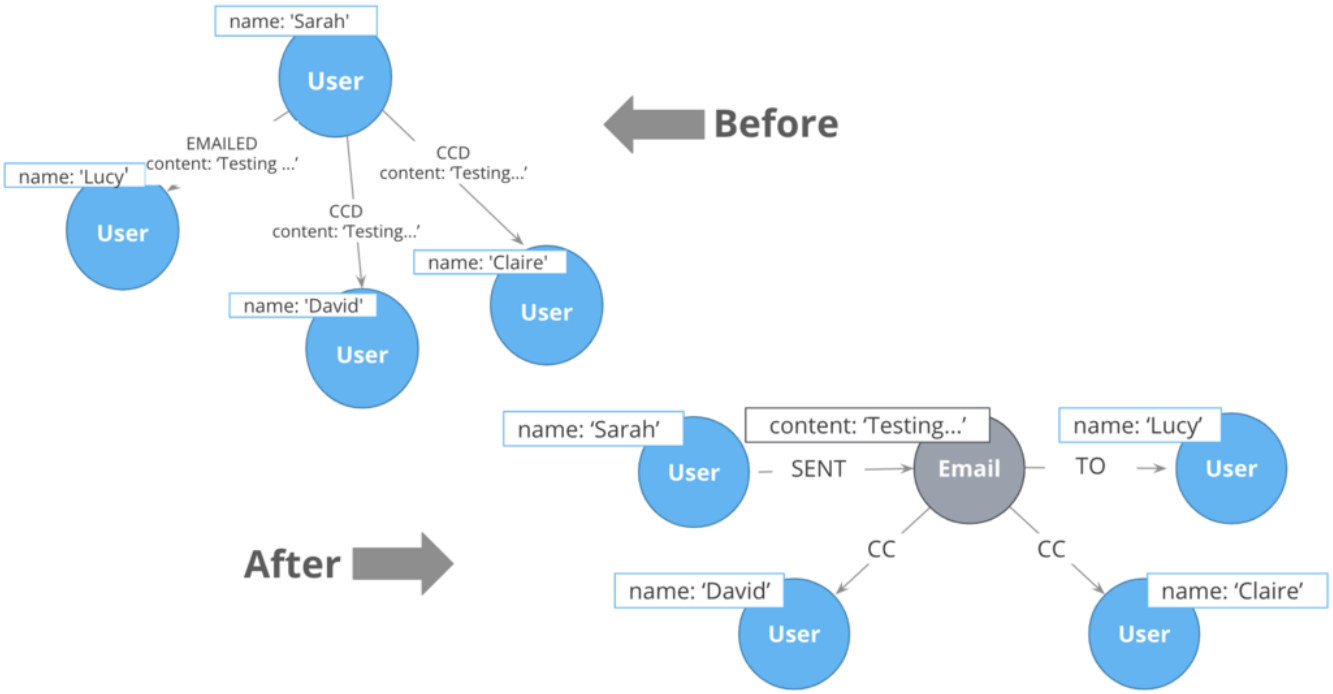
In this example, we replace the *WORKED_AT* relationship with an Employment intermediate node. This provides a connection point that allows us connect any amount of information to Patrick's term of employment at Acme.

Intermediate nodes: sharing context



This model allows different employment events to share contextual information. Person nodes can have a shared Role or Company, and allow us to very easily trace either the full details of a single person's career, or the overlap between different individuals.

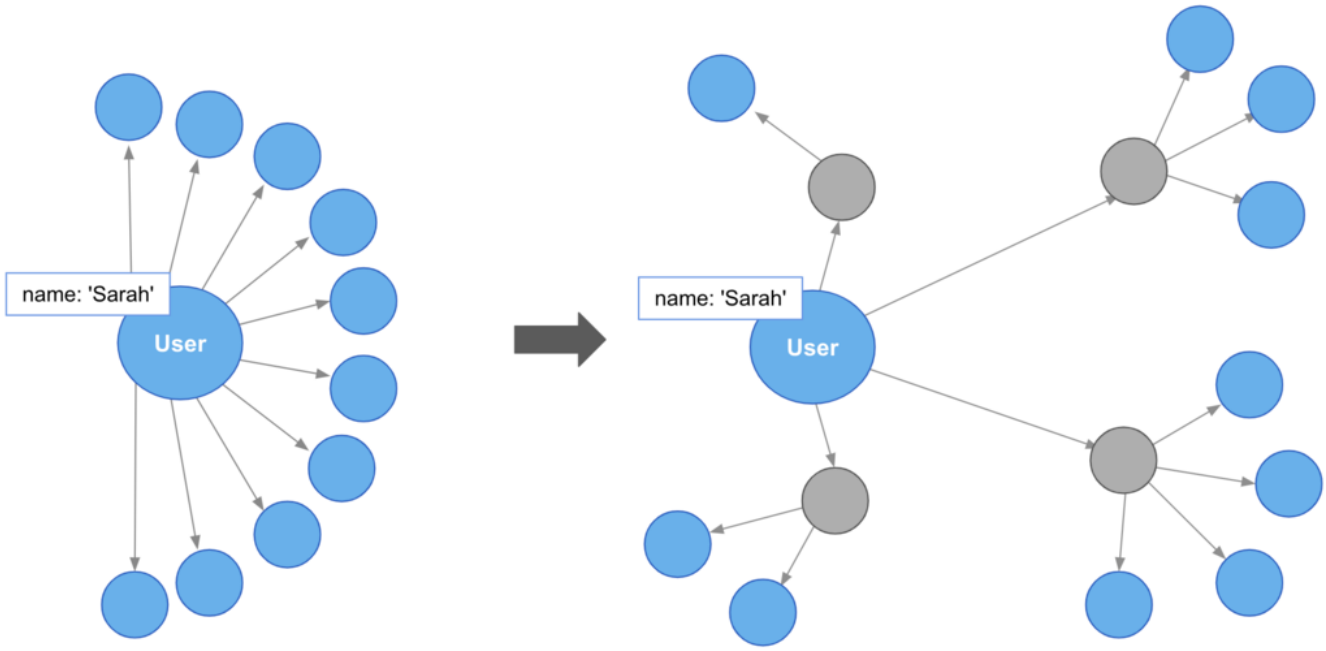
Intermediate nodes: sharing data



Intermediate nodes also allow you to deduplicate information. We mentioned a similar principle when we discussed fanout where splitting a property onto its own node allows you to reference it via a relationship instead of a repeated property.

In this case, the Email intermediate node spares us having to repeat the email content on every single relationship.

Intermediate nodes: organizing data



In addition, intermediate nodes can be invaluable organizing structures. This is a more expansive zoomed-out view of the same data as the previous example. The grey nodes are the Email intermediate nodes.

When we model Email as a relationship, as on the left, we encounter two problems. First, Sarah's node becomes extremely dense. For every single recipient of any message she writes, she gains another relationship. On the right, she only has one relationship per email, regardless of the number of recipients. Dense nodes can be tricky to work with.

Second, it is difficult and expensive to determine who in Sarah's recipient network has received a given message. In the left model, we need to traverse every relationship and check the email content to see which messages are shared. This specific case requires between six and nine "wasted" hops, depending on whether we are looking for the four-recipient message or the one-recipient message. On the right, with the Email intermediate node, we still need to check every message. But having done so, we can guarantee that every node connected to that email is a relevant one. There will only ever be three "wasted" hops.

Linked lists

Linked lists are useful whenever the sequence of objects matters. The typical pattern is to define relationships between sequential items with the names NEXT or PREVIOUS, although those words are not verbs. As always, choose one or the other based on the questions you expect to ask.



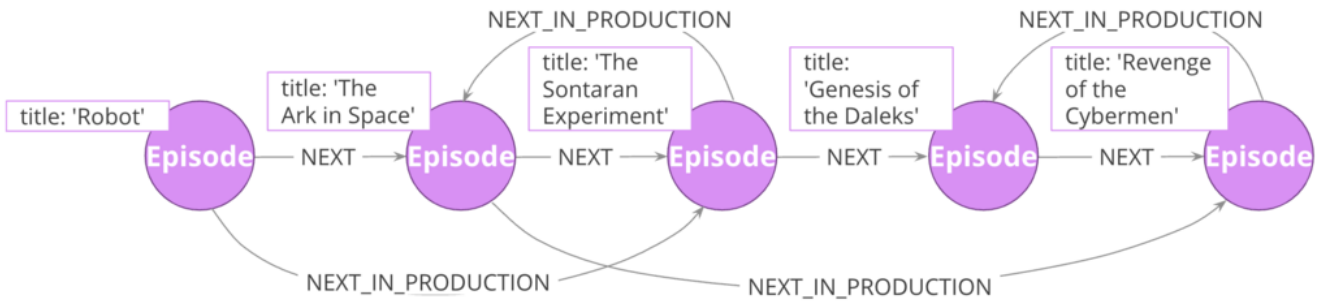


Linked lists are not unique to graphs. They are a common pattern in lots of data modeling paradigms. In some of those paradigms, you see both the singly-linked list (above) and the doubly-linked list (below).

You should **never** use a doubly-linked list in Neo4j because doubly-linked lists use redundant symmetrical relationships.

Interleaved linked list

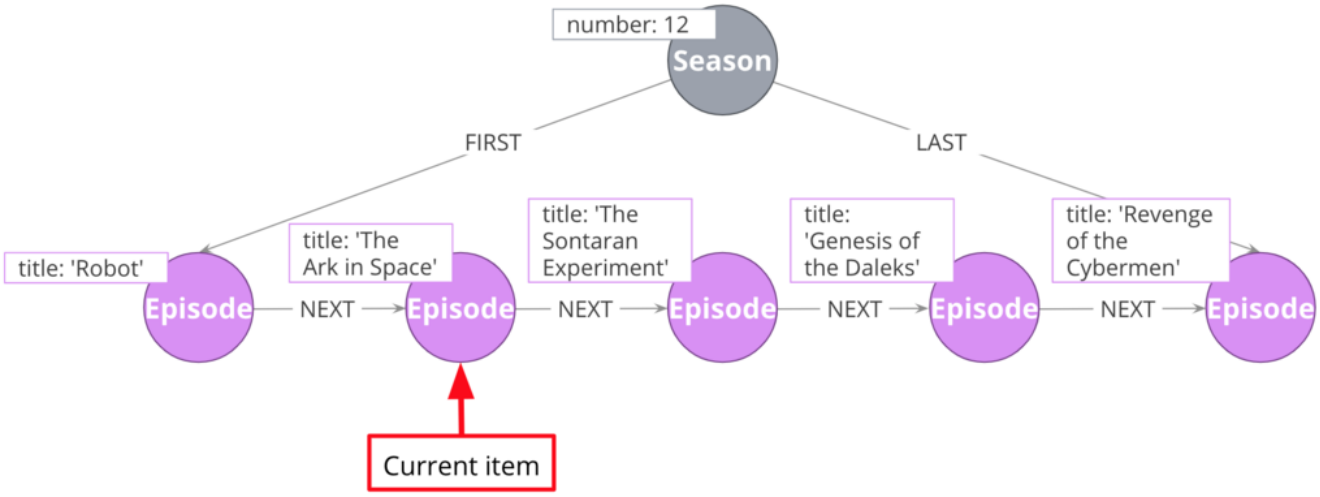
You can add complexity to linked lists. Sometimes, there are multiple ways you could sequence a set of items.



For example, the order in which TV show episodes are shown is not always the same order in which they were created. Here, we again have the episodes of Dr. Who season 12, but with two sequences: the order of airing (NEXT) and the order of production (NEXT_IN_PRODUCTION).

Head and tail of linked list

Here is another pattern that uses a linked list. When working with linked list, there is generally a "parent" node that is used as the entry point. That parent almost always points to the first item in the sequence, using an appropriately named relationship. It may also point to the last item. For cases where you are tracking progress through the sequence, the parent node may also have a pointer to the "current" item.



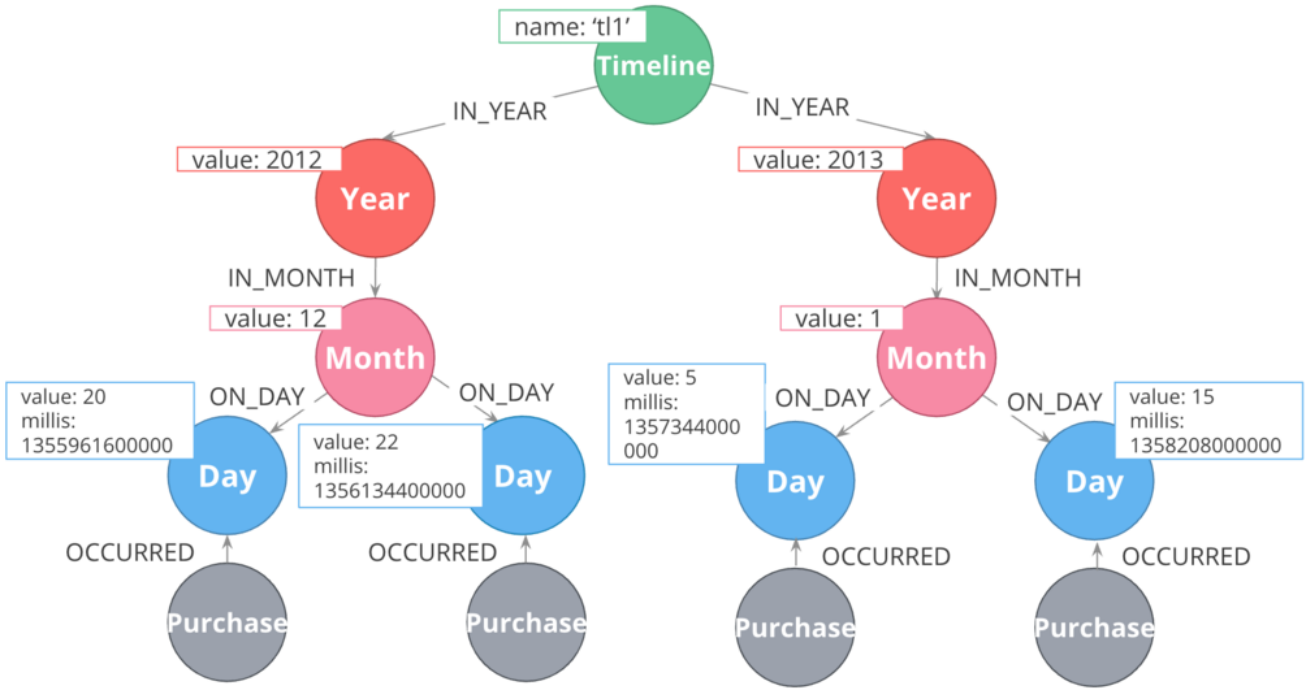
Some possible use cases:

- Add episodes as they are broadcast.
- Maintain pointer to first and last episodes.
- Find all episodes broadcast so far.
- Find latest episode broadcast so far.

Timeline tree

Trees are not complicated, but you will learn about one special kind of tree, the timeline tree. Timeline trees can be a useful structure when you want to use time as either an anchor or a navigational aid in your querying, and when the periods of time that interest you vary.

The topmost node in the timeline is an “all time” node. That then subdivides into whatever time periods interest you.



Here, we have chosen years, months, and days; we could just have equally chosen quarters, weeks, hours, minutes, or anything else. All of our "data" nodes then link into the timeline tree at the appropriate leaf node. Here, we have connected every purchase to a day.

With this structure, some otherwise-expensive time questions become easy. For example, finding all purchases in a given time periods, where the period of interest might be a day, a month, or a year, depending on circumstances. Simply anchor on the timeline at the appropriate time period of interest, then traverse downward through the tree.

What is more interesting, though, is dealing with time when time is part of the traversal, but not the anchor. For example, looking for all purchases that happened within a some time period relative to another event. You anchor on the specified event, traverse into the timeline tree, traverse to the appropriate time period of interest, then traverse downward. Without a timeline structure in the graph, this kind of query frequently involves a lot of property lookups and inefficient gather-and-inspect.

NOTE

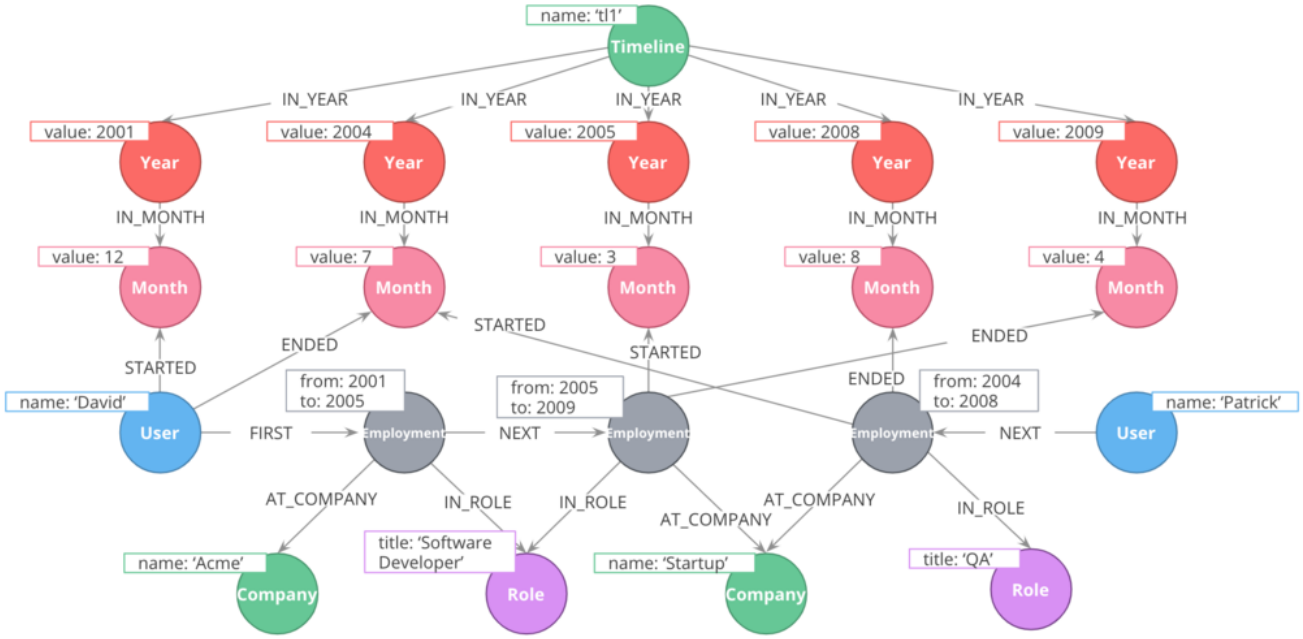
That is not to say that timeline trees are the go-to "best" method of expressing time! Quite the opposite—timeline trees consume a lot of space relative to the data they contain. Most often, even when time is an important part of the answer to a question, a simple property is sufficient.

But we are sharing this structure with you so that, in cases where a property alone is not good enough, you do not need to reinvent the concept of a timeline tree yourself.

Using multiple structures

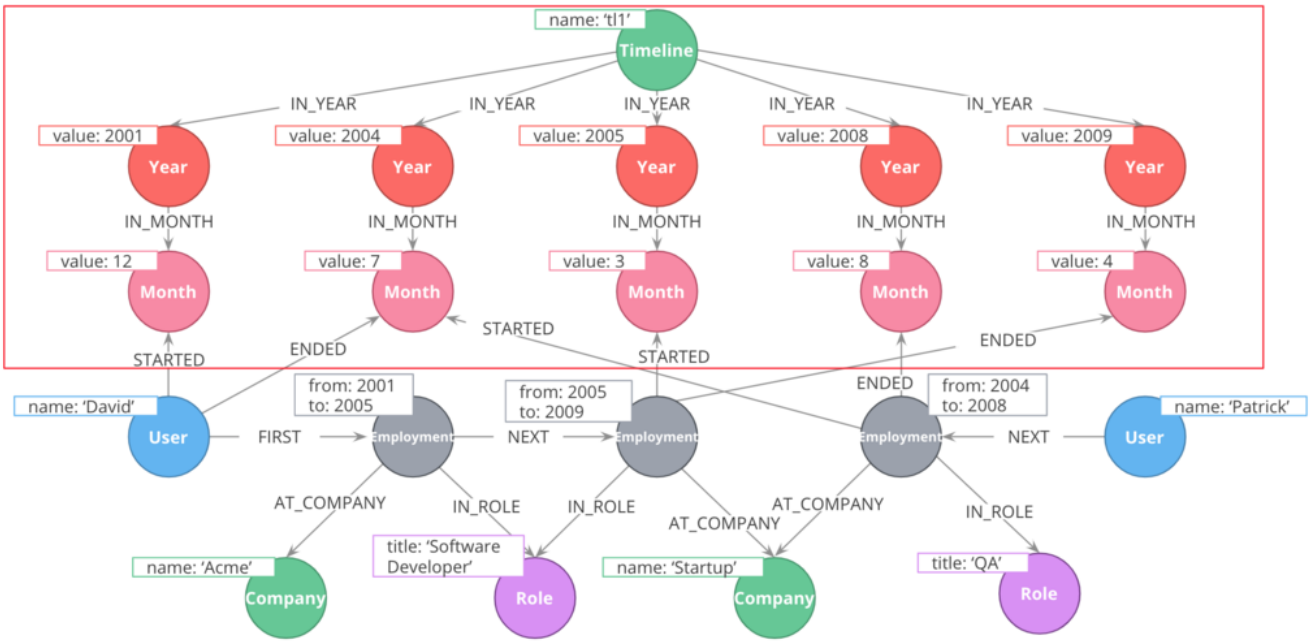
All of the structures you have learned about thus far are not entire models in their own right. They are tiny patterns with specific use cases. Modeling questions as a whole are usually larger and more complex than that. Frequently, you will see multiples of these structures used together in order to fully solve a problem.

Here is one example.



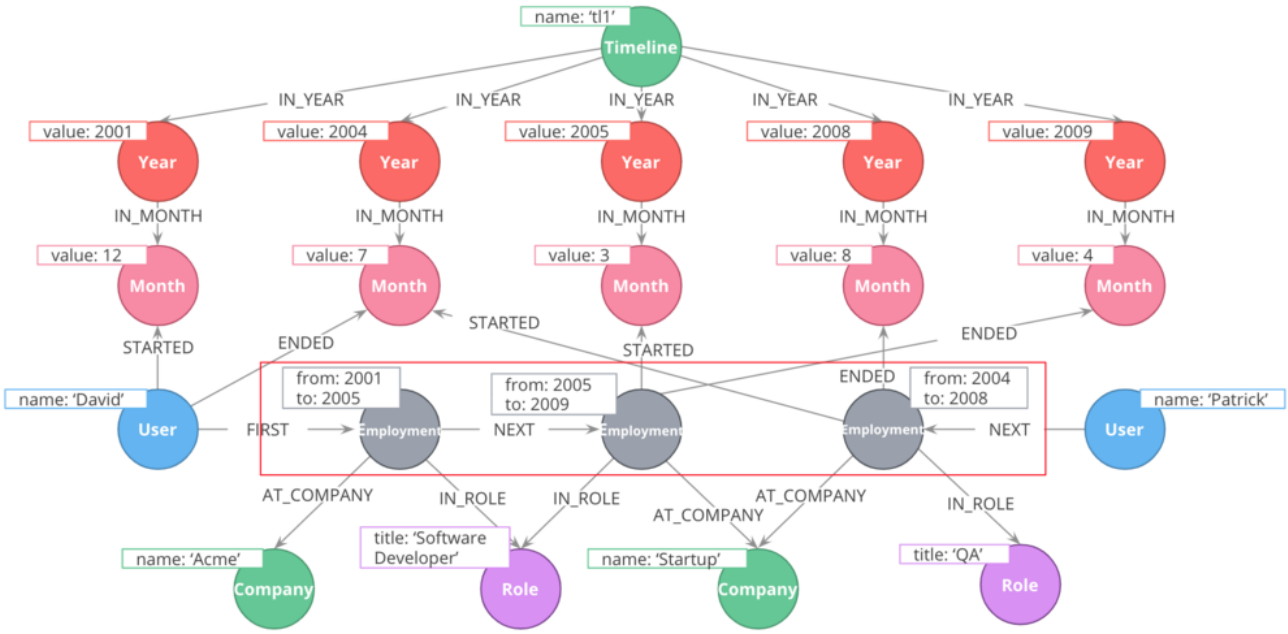
Using the timeline tree

At the top, we have a timeline tree that subdivides into years and months.

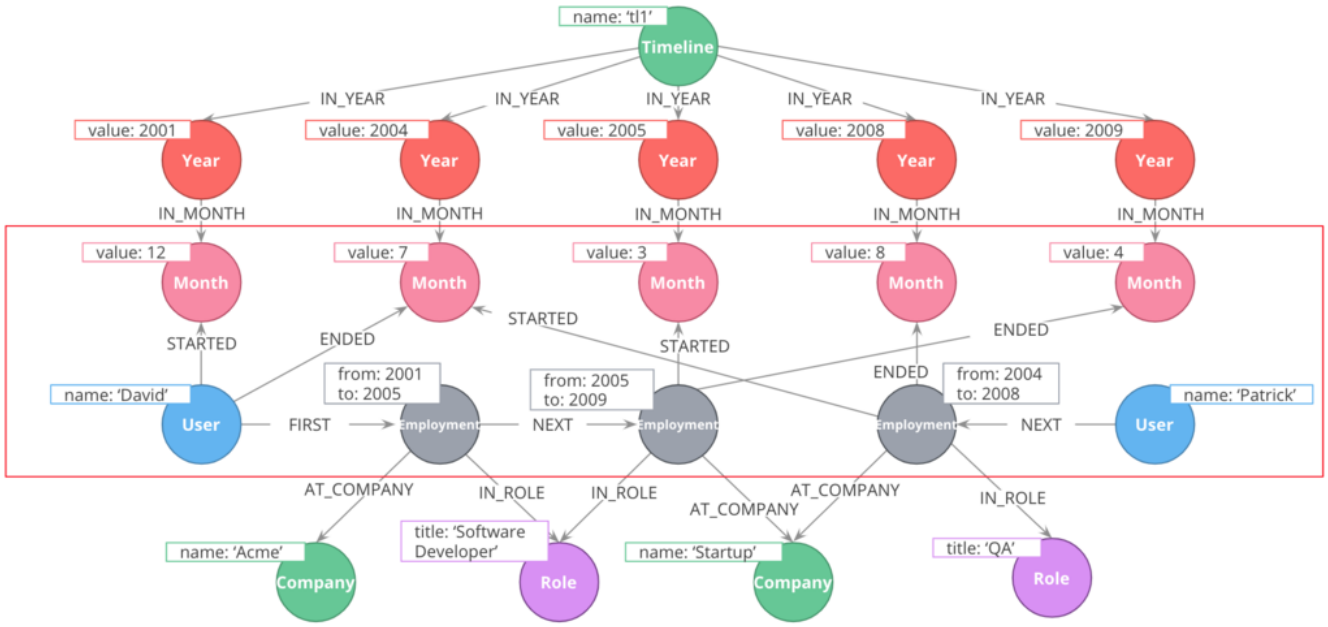


Using intermediate nodes

The relevant items connected to the timeline are Employment events, which link to the starting and ending months of that employment. Employment events themselves are intermediate nodes connecting people, companies, and job roles. In this case, the intermediate nodes are serving the dual purposes of reducing the density of Person nodes, and providing an attachment point for the timeline tree.



Using linked list



Furthermore, the Employment events themselves are organized into linked lists for each person. This allows us to represent the sequence of employment events in an individual's career.

Exercise 4: Model a Learning Management System (LMS)

Given a description of the domain, sample data, and the application questions:

1. Create the model (entities and connections) using the Arrow tool.
    - Look for any use cases that will use some common patterns you have learned about.
2. Add sample data to the model using the Arrow tool and confirm questions can be answered using the model.

Exercise 4 instructions: The domain

- There are many courses in the LMS, each of which contains a number of lessons that must be completed in a specific order.
- Every course grants a certificate upon completion.
- This certificate has a term of validity. When it expires, students must take the course again.
- Students can enroll in as many simultaneous courses as they want to.
- When a student logs in and chooses a course, the LMS must send them to their latest unfinished lesson.

Exercise 4 instructions: Sample data

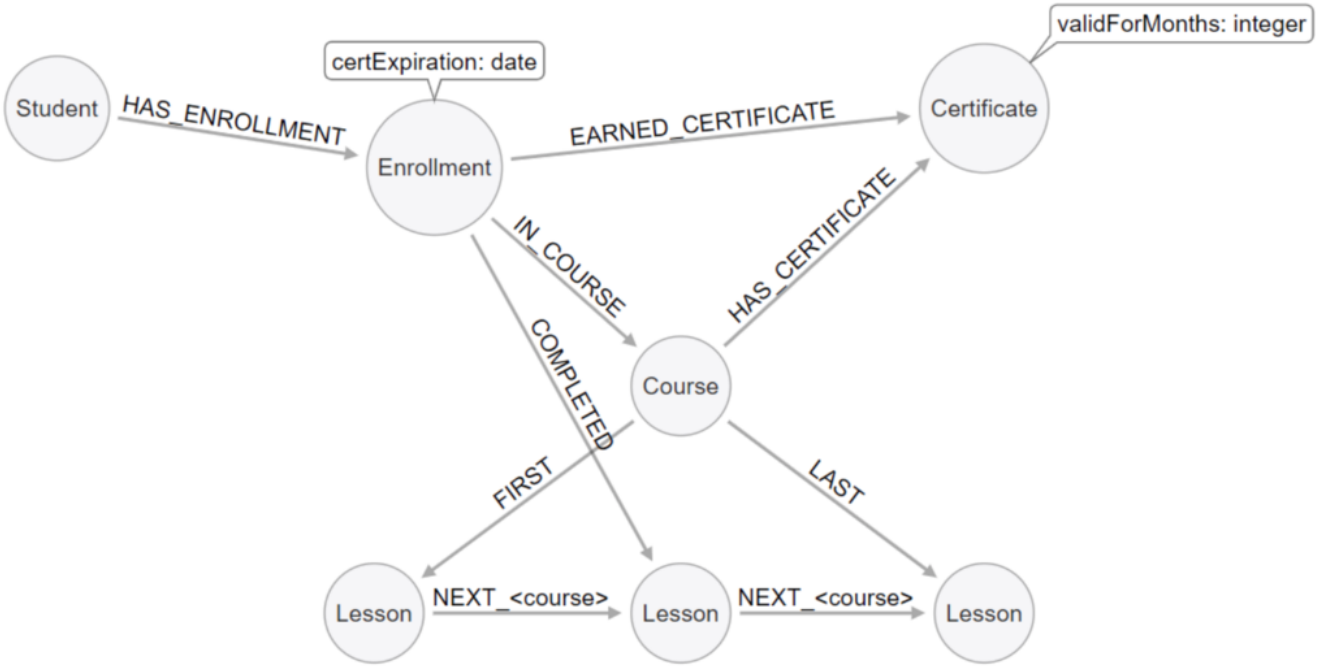
Courses	Lessons	Certificate
Introduction to Neo4j	Graph Theory, Graph Databases, Basic Cypher	2-year validity
Neo4j for Developers	Graph Theory, Property Graph, Graph Databases	6-month validity

Students	Completed Courses	In-Progress Courses
Alice	Introduction to Neo4j (2016), Introduction to Neo4j (2018)	Introduction to Neo4j (lesson 1)
Dan		Introduction to Neo4j (lesson 3), Neo4j for Developers (lesson 2)

Exercise 4 instructions: Application questions

1. Which lesson(s) is Dan currently working on?
2. What are Alice's current certifications?
3. Which lessons are in the Neo4j for Developers course?
4. What is the last lesson in the Introduction to Neo4j course?
5. Which lesson follows Graph Theory in the Neo4j for Developers course?
6. Who has completed Introduction to Neo4j?

Exercise 4 solution: Application model



The Course and Lessons are structured as a linked list with a parent. We include the course name in the NEXT relationship, because some lessons are used by multiple courses, but in a different order. In other words, we have an interleaved linked list, and need to distinguish the different sequences.

Courses are associated with a Certificate, which contains, among other things, a term of validity expressed here in months.

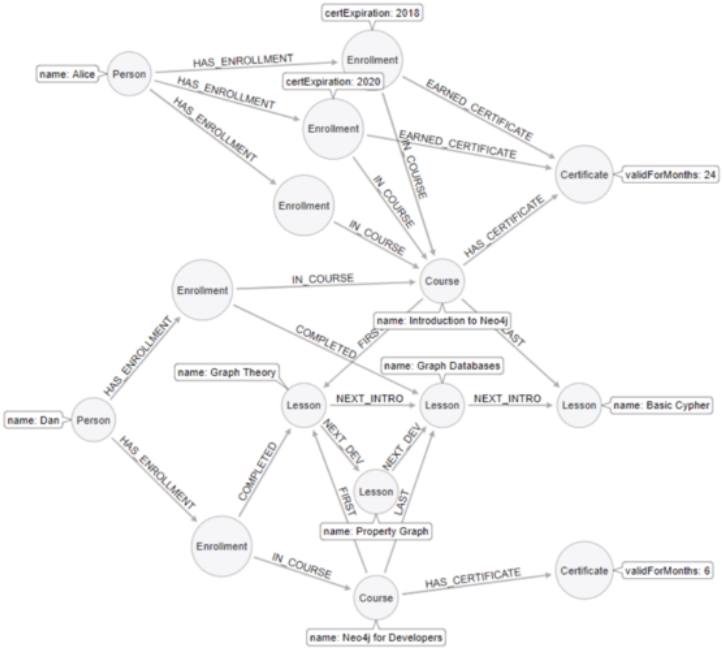
Students do not directly interact with courses or lessons. Instead, we use an Enrollment intermediate node to organize their interactions. Enrollments are linked to a single course and contain a pointer to which lesson was most recently completed. When a COMPLETED relationship is created with the lesson marked LAST in a given course, that is the trigger to create an EARNED_CERTIFICATE relationship. At this point, the certExpiration property will be populated by adding the term of validity to the current timestamp. This intermediate node allows us to keep track of students' progress through different concurrent courses, and to differentiate multiple subsequent passes through the same course.

Here is the markup for the model:

```
<ul class="graph-diagram-markup" data-internal-scale="0.67" data-external-
scale="1">
  <li class="node" data-node-id="0" data-x="-550.4655813128935" data-y="-
1216.8149073094257">
    <span class="caption">Student</span>
  </li>
  <li class="node" data-node-id="1" data-x="592.2255987630683" data-y="-
808.4017464284152">
    <span class="caption">Enrollment</span><dl
class="properties"><dt>certExpiration</dt><dd>date</dd></dl></li>
  <li class="node" data-node-id="2" data-x="1530.5288281421945" data-y="-
188.47328130455">
    <span class="caption">Course</span>
  </li>
  <li class="node" data-node-id="3" data-x="696.1411792240782" data-
```

```
y="475.60253854551536">
    <span class="caption">Lesson</span>
  </li>
  <li class="node" data-node-id="4" data-x="1562.4419933343002" data-
y="475.60253854551536">
    <span class="caption">Lesson</span>
  </li>
  <li class="node" data-node-id="5" data-x="2419.99523773733" data-
y="475.60253854551536">
    <span class="caption">Lesson</span>
  </li>
  <li class="node" data-node-id="6" data-x="2459.771940162705" data-y="-
808.4017464284152">
    <span class="caption">Certificate</span><dl
class="properties"><dt>validForMonths</dt><dd>integer</dd></dl></li>
  <li class="relationship" data-from="4" data-to="5">
    <span class="type">NEXT_&lt;course&gt;</span>
  </li>
  <li class="relationship" data-from="3" data-to="4">
    <span class="type">NEXT_&lt;course&gt;</span>
  </li>
  <li class="relationship" data-from="2" data-to="3">
    <span class="type">FIRST</span>
  </li>
  <li class="relationship" data-from="2" data-to="5">
    <span class="type">LAST</span>
  </li>
  <li class="relationship" data-from="0" data-to="1">
    <span class="type">HAS_ENROLLMENT</span>
  </li>
  <li class="relationship" data-from="1" data-to="2">
    <span class="type">IN_COURSE</span>
  </li>
  <li class="relationship" data-from="2" data-to="6">
    <span class="type">HAS_CERTIFICATE</span>
  </li>
  <li class="relationship" data-from="1" data-to="6">
    <span class="type">EARNED_CERTIFICATE</span>
  </li>
  <li class="relationship" data-from="1" data-to="4">
    <span class="type">COMPLETED</span>
  </li>
</ul>
```

Exercise 4 solution: Sample data



1. Which lesson(s) is Dan currently working on?
2. What are Alice's current certifications?
3. Which lessons are in the **Neo4j for Developers** course?
4. What is the last lesson in the **Introduction to Neo4j** course?
5. Which lesson follows **Graph Theory** in the **Neo4j for Developers** course?
6. Who has completed **Introduction to Neo4j**?

Here is the sample data fully fleshed out. Use the below markup to get it into Arrows.

```
<ul class="graph-diagram-markup" data-internal-scale="0.21" data-external-
scale="1">
  <li class="node" data-node-id="0" data-x="-396.2973779224105" data-y="-
1598.542651644388">
    <span class="caption">Person</span><dl
class="properties"><dt>name</dt><dd>Alice</dd></dl></li>
  <li class="node" data-node-id="1" data-x="696.1411792240782" data-y="-
775.5184522555546">
    <span class="caption">Enrollment</span>
  </li>
  <li class="node" data-node-id="2" data-x="1562.4419933343002" data-y="-
301.66353654081996">
    <span class="caption">Course</span><dl
class="properties"><dt>name</dt><dd>Introduction to Neo4j</dd></dl></li>
  <li class="node" data-node-id="3" data-x="696.1411792240782" data-
y="475.60253854551536">
    <span class="caption">Lesson</span><dl
class="properties"><dt>name</dt><dd>Graph Theory</dd></dl></li>
  <li class="node" data-node-id="4" data-x="1562.4419933343002" data-
y="475.60253854551536">
    <span class="caption">Lesson</span><dl
class="properties"><dt>name</dt><dd>Graph Databases</dd></dl></li>
  <li class="node" data-node-id="5" data-x="2419.99523773733" data-
y="475.60253854551536">
    <span class="caption">Lesson</span><dl
class="properties"><dt>name</dt><dd>Basic Cypher</dd></dl></li>
  <li class="node" data-node-id="6" data-x="2452.0596471356525" data-y="-
970.0863461358005">
    <span class="caption">Certificate</span><dl
class="properties"><dt>validForMonths</dt><dd>24</dd></dl></li>
  <li class="node" data-node-id="7" data-x="879.8886791627793" data-y="-
1253.5663862086055">
    <span class="caption">Enrollment</span><dl
```

```
class="properties"><dt>certExpiration</dt><dd>2020</dd></dl></li>
  <li class="node" data-node-id="8" data-x="1124.3637964049385" data-y="-
1683.353885999956">
    <span class="caption">Enrollment</span><dl
class="properties"><dt>certExpiration</dt><dd>2018</dd></dl></li>
  <li class="node" data-node-id="9" data-x="-591.3704658622172" data-y="-
2730.474279887639"></li>
  <li class="node" data-node-id="10" data-x="-620.4774799631609" data-
y="565.4384413761886">
    <span class="caption">Person</span><dl
class="properties"><dt>name</dt><dd>Dan</dd></dl></li>
  <li class="node" data-node-id="11" data-x="1148.4184037393607" data-
y="1678.755606466262">
    <span class="caption">Course</span><dl
class="properties"><dt>name</dt><dd>Neo4j for Developers</dd></dl></li>
  <li class="node" data-node-id="12" data-x="1148.4184037393607" data-
y="1015.6712674382908">
    <span class="caption">Lesson</span><dl
class="properties"><dt>name</dt><dd>Property Graph</dd></dl></li>
  <li class="node" data-node-id="13" data-x="-81.87263599082607" data-
y="1955.3561577156422"></li>
  <li class="node" data-node-id="14" data-x="61.062009896806536" data-y="-
246.0263961108761">
    <span class="caption">Enrollment</span>
  </li>
  <li class="node" data-node-id="15" data-x="181.74861463859963" data-
y="1357.3756722976887">
    <span class="caption">Enrollment</span>
  </li>
  <li class="node" data-node-id="16" data-x="2452.0596471356525" data-
y="1507.105178106398">
    <span class="caption">Certificate</span><dl
class="properties"><dt>validForMonths</dt><dd>6</dd></dl></li>
  <li class="relationship" data-from="4" data-to="5">
    <span class="type">NEXT_INTRO</span>
  </li>
  <li class="relationship" data-from="3" data-to="4">
    <span class="type">NEXT_INTRO</span>
  </li>
  <li class="relationship" data-from="2" data-to="3">
    <span class="type">FIRST</span>
  </li>
  <li class="relationship" data-from="2" data-to="5">
    <span class="type">LAST</span>
  </li>
  <li class="relationship" data-from="0" data-to="1">
    <span class="type">HAS_ENROLLMENT</span>
  </li>
  <li class="relationship" data-from="1" data-to="2">
    <span class="type">IN_COURSE</span>
  </li>
  <li class="relationship" data-from="2" data-to="6">
    <span class="type">HAS_CERTIFICATE</span>
  </li>
  <li class="relationship" data-from="0" data-to="7">
    <span class="type">HAS_ENROLLMENT</span>
```

```
  </li>
  <li class="relationship" data-from="0" data-to="8">
    <span class="type">HAS_ENROLLMENT</span>
  </li>
  <li class="relationship" data-from="7" data-to="2">
    <span class="type">IN_COURSE</span>
  </li>
  <li class="relationship" data-from="8" data-to="2">
    <span class="type">IN_COURSE</span>
  </li>
  <li class="relationship" data-from="8" data-to="6">
    <span class="type">EARNED_CERTIFICATE</span>
  </li>
  <li class="relationship" data-from="7" data-to="6">
    <span class="type">EARNED_CERTIFICATE</span>
  </li>
  <li class="relationship" data-from="3" data-to="12">
    <span class="type">NEXT_DEV</span>
  </li>
  <li class="relationship" data-from="12" data-to="4">
    <span class="type">NEXT_DEV</span>
  </li>
  <li class="relationship" data-from="11" data-to="3">
    <span class="type">FIRST</span>
  </li>
  <li class="relationship" data-from="11" data-to="4">
    <span class="type">LAST</span>
  </li>
  <li class="relationship" data-from="10" data-to="14">
    <span class="type">HAS_ENROLLMENT</span>
  </li>
  <li class="relationship" data-from="10" data-to="15">
    <span class="type">HAS_ENROLLMENT</span>
  </li>
  <li class="relationship" data-from="15" data-to="11">
    <span class="type">IN_COURSE</span>
  </li>
  <li class="relationship" data-from="14" data-to="2">
    <span class="type">IN_COURSE</span>
  </li>
  <li class="relationship" data-from="15" data-to="3">
    <span class="type">COMPLETED</span>
  </li>
  <li class="relationship" data-from="14" data-to="4">
    <span class="type">COMPLETED</span>
  </li>
  <li class="relationship" data-from="11" data-to="16">
    <span class="type">HAS_CERTIFICATE</span>
  </li>
</ul>
```

Check your understanding

Question 1

What graph modeling pattern can you use to add more contextual and organizational information to relationships that is not easily modeled with simple relationship properties?

Select the correct answer.

- ☐ Sub-relationships.
- ☐ Linked list.
- ☐ Timeline tree.
- ☑ Intermediate nodes.

Question 2

In Neo4j, which structure is not recommended as a best practice for representing linked lists?

Select the correct answer.

- ☐ Interleaved linked lists with multiple starting points for navigation.
- ☑ Doubly linked lists.
- ☐ Single linked list.
- ☐ Linked list with nodes that point to the head and tail of the list.

Question 3

What common structure is used to model data where you need to know when an event occurred in the application and find other events that occurred in the same time-frame?

Select the correct answer.

- ☐ Linked list where each node has a timestamp.
- ☐ Intermediate nodes where the intermediate nodes have time-related data.
- ☑ Timeline tree
- ☐ Doubly-linked list where the symmetric relationbship is the timestamp relationship.

Summary

You should now be able to:

- Describe common graph structures used in modeling:
  - Intermediate nodes
  - Linked lists
  - Timeline trees
  - Multiple structures in a single graph

Refactoring and Evolving a Model

About this module

At the end of this module, you should be able to:

- Describe why you would refactor a graph data model.
- Refactor a model to:
  - Eliminate duplicate data in nodes.
  - Use node labels rather than properties.
  - Extract property values to create nodes.

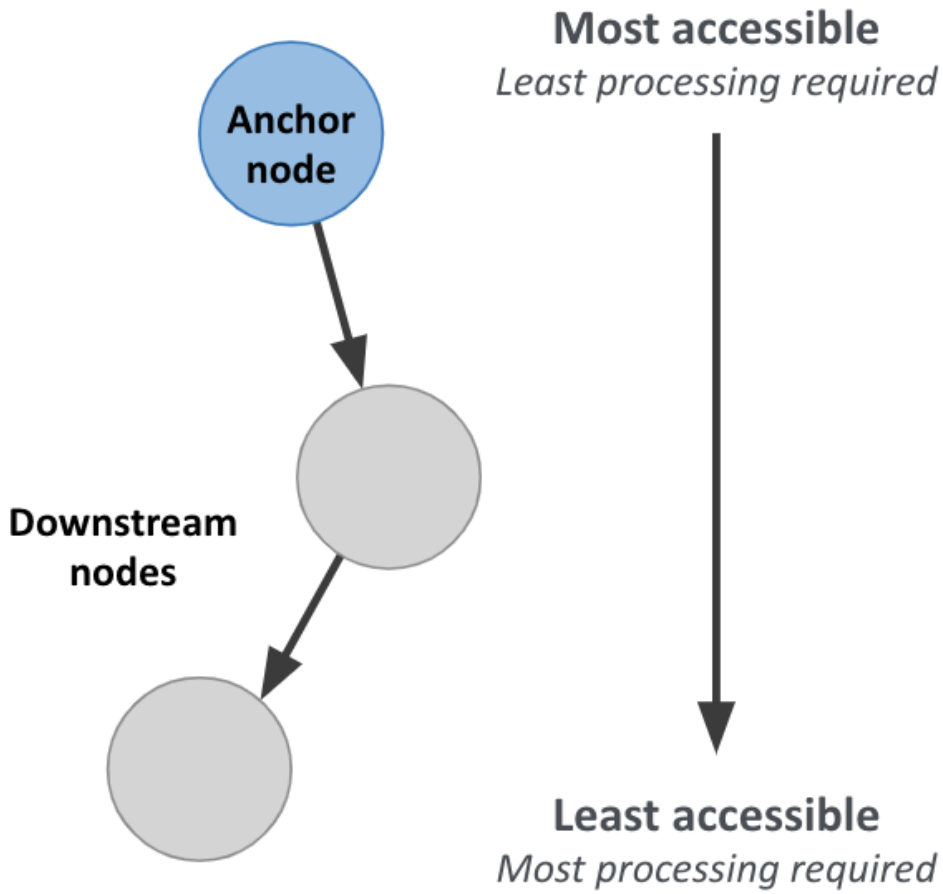
What is refactoring?

Refactoring is the process of changing the data structure without altering its semantic meaning.

- Most of the time, refactoring simply involves moving data from one structure to another within the graph.
- In some cases, refactoring involves adding more data from other sources.

The most common type of refactoring of a graph is to use a property value to restructure the graph. That is, a property value is used to create a label, a node, or a relationship.

Review: Hierarchy of Accessibility



For each data object, how much work must Neo4j do to evaluate if this is a "good" path or a "bad" one?

1. Anchor node label, indexed anchor node properties
2. Relationship types
3. Non-indexed anchor node properties
4. Downstream node labels
5. Relationship properties, downstream node properties

Recall the hierarchy of accessibility. In general, refactoring involves moving data around this hierarchy, in order to better answer a question.

Why refactor?

Data models can be optimized for one of four things:

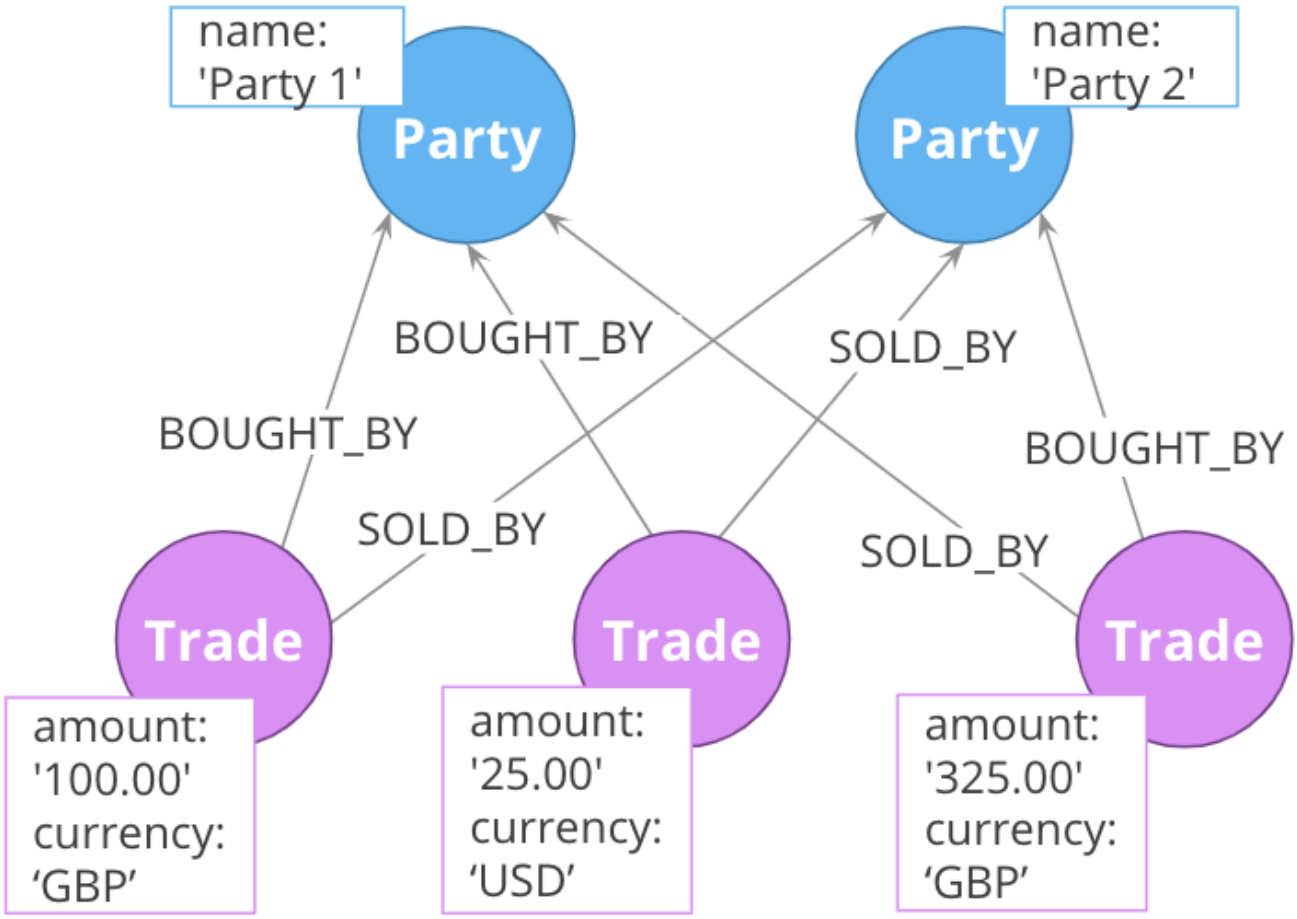
- Query performance
- Model simplicity & intuitiveness
- Query simplicity (i.e., simpler Cypher strings)
- Easy data updates

NOTE Improving behavior in one of these areas frequently involves sacrifices in others.

Another **important** reason to refactor is to accommodate new application questions in the same model.

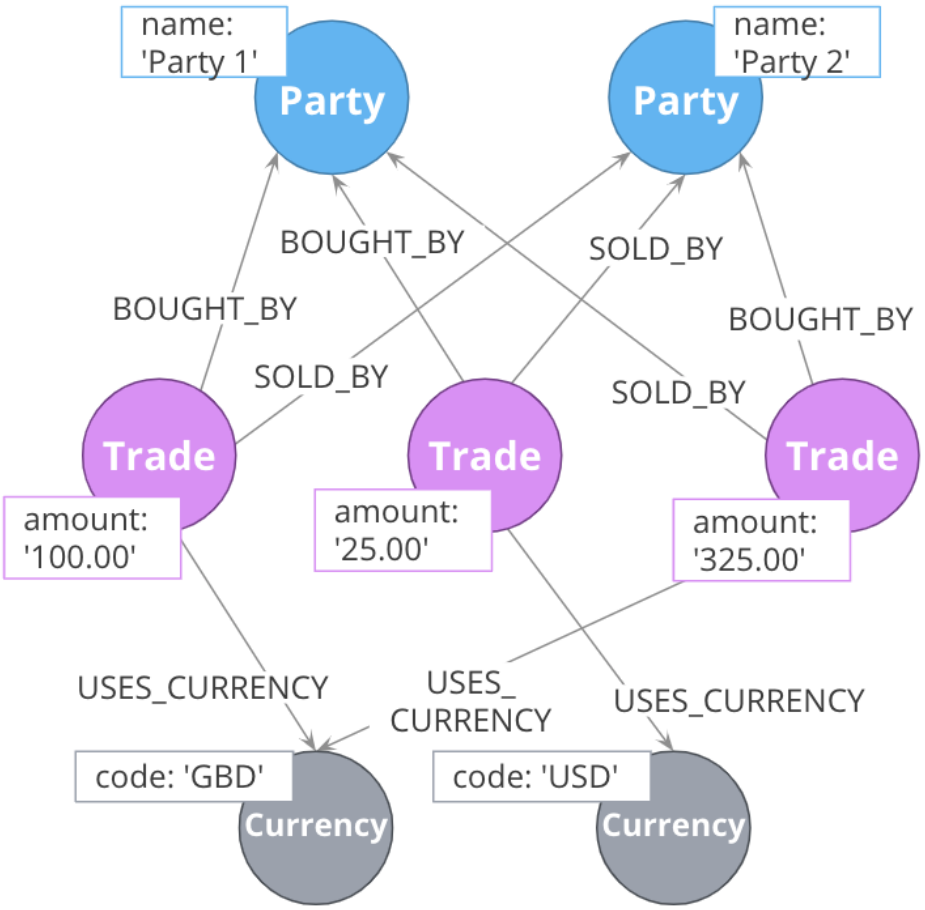
Goal: Eliminate duplicate data in properties

There may be data that is duplicated in many nodes in your graph. In this example the type of currency is a property of a node.



What if we do not want to duplicate the currency type data in all of our Trade nodes?

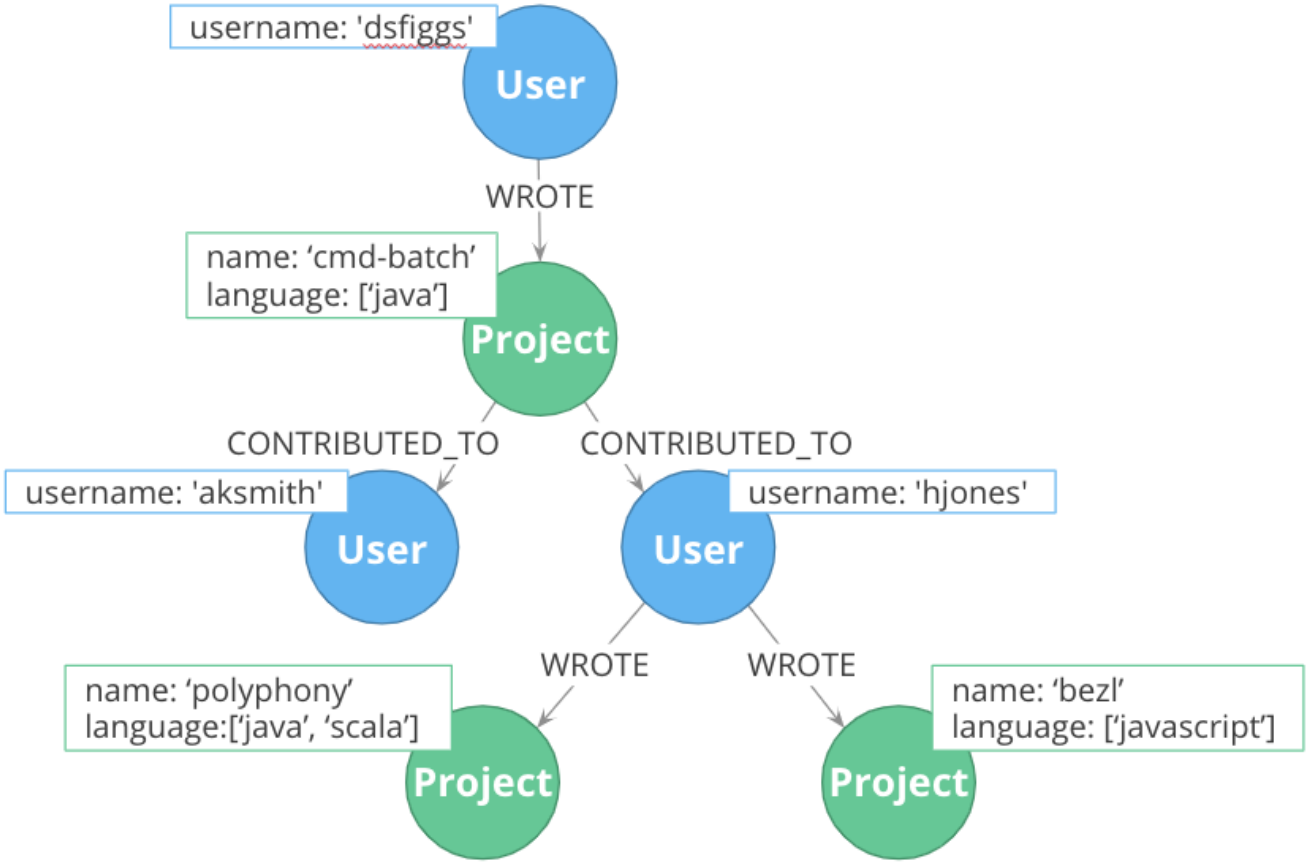
Refactor example: Extracting nodes from properties



With the Currency node label, we can easily answer questions such as how many Trades use the USD currency without having to gather this information from the Trade nodes.

Goal: Use labels instead of property values

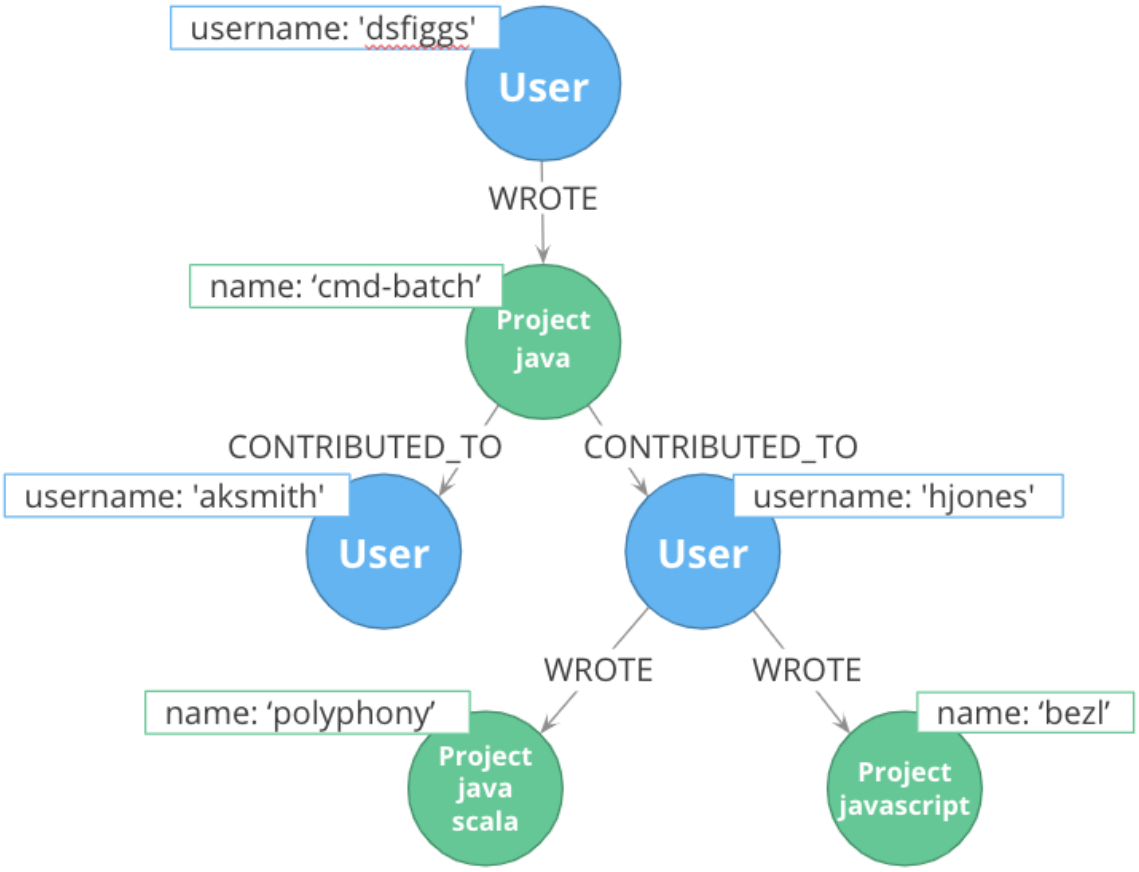
If many nodes in the model contain the same value for a property, another solution is the use the property value as a label.



In this example, you see that the Project nodes have a property named *language* where the property could contain multiple languages. What if you wanted to query the graph to find all projects that are written in *java*? To do this, you would need to retrieve all of the Project nodes and look for *java* in the language list for each node. This is an expensive query on a large graph.

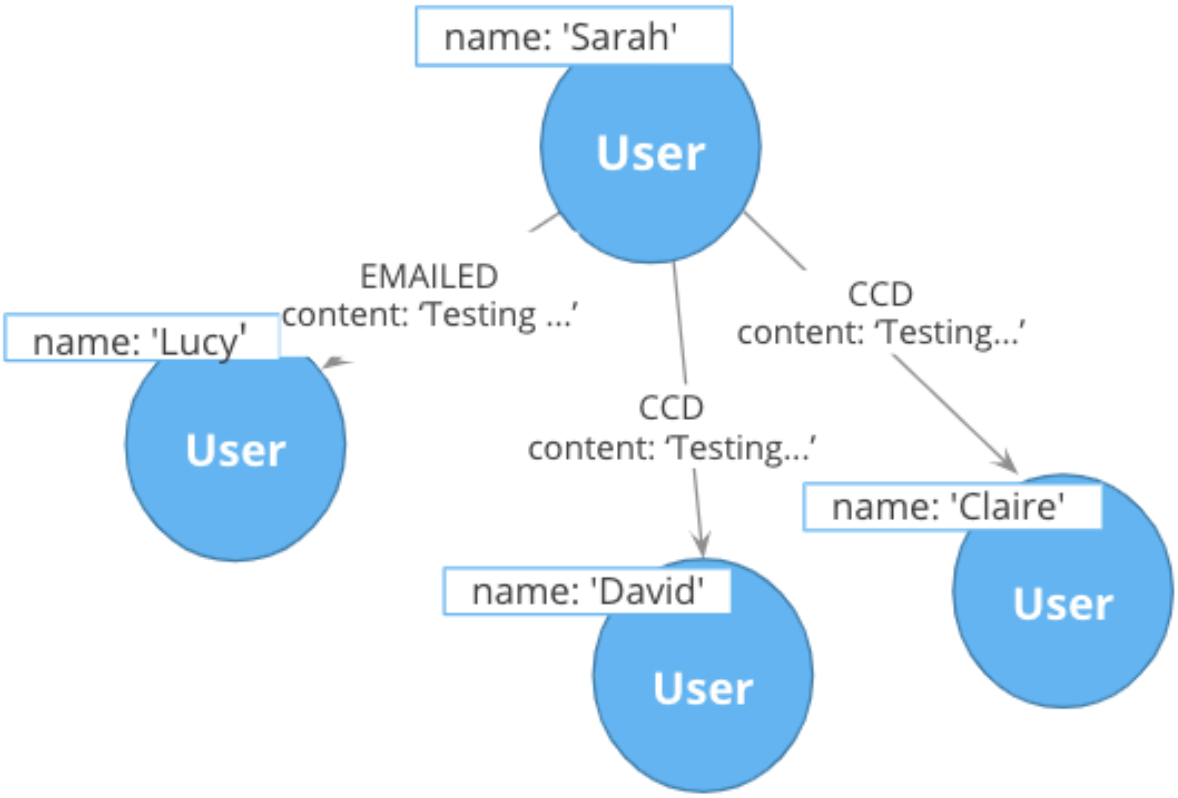
Refactor example: Turn property values into labels for nodes

In this scenario, we would go through all Project nodes in the graph and relabel the Project nodes based upon the values in the *language* list. Then we eliminate completely the *language* property.



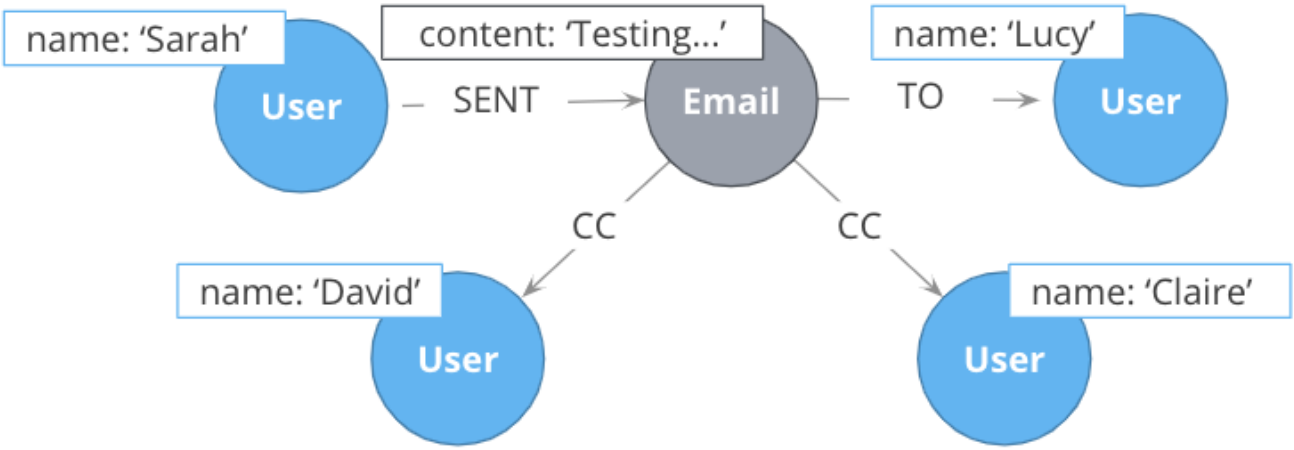
Goal: Use nodes instead of properties for relationships

Previously in this training, you learned that intermediate nodes is sometimes a best practice. Here is the example we saw earlier with the content of the email as a property for the relationship.



Refactor: Extract nodes from relationship properties

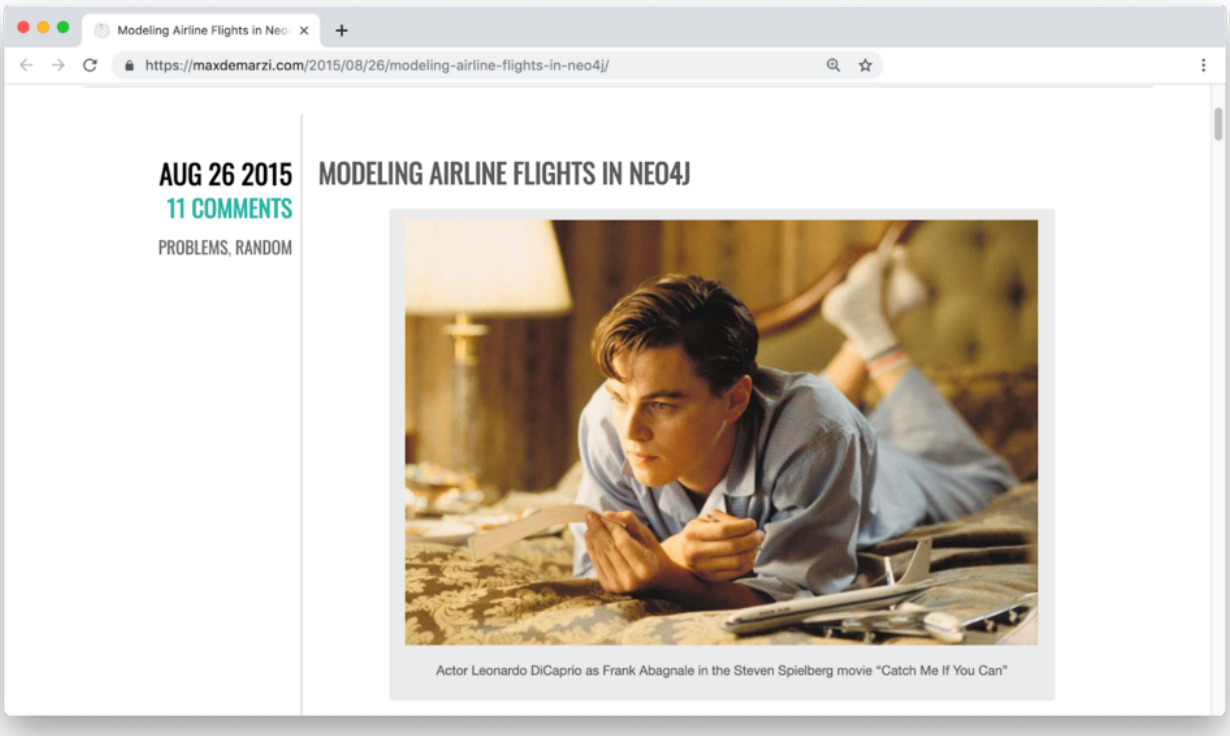
Here is what the graph would look like after we went through all EMAILED and CCD relationships to create the intermediate nodes for the content.



Intermediate nodes are also a frequent result of refactoring. What was previously a relationship property is now a node property, which is not necessarily any more accessible from a query processing standpoint. But it makes the model simpler by reducing the number of relationships connected to each Person node.

Notice that not only is the new node with the label *Email*, but also relationship types are recreated with different types.

Refactoring example: Modeling airline flights



Here is a very good example of how to model and refactor a model.

Credit: Max De Marzi https://maxdemarzi.com/2015/08/26/modeling-airline-flights-in-neo4j/

We will now walk through an iterative process of graph development and refactoring. We will use, as an example, modeling airline flights for an Orbitz-style use case: that is, answering the problem, “I want to fly from Malmo to New York on Friday”.

You can read in depth about this on Max’s blog.

Initial question for our model

Question: What flights will take me from Malmo to New York on Friday?

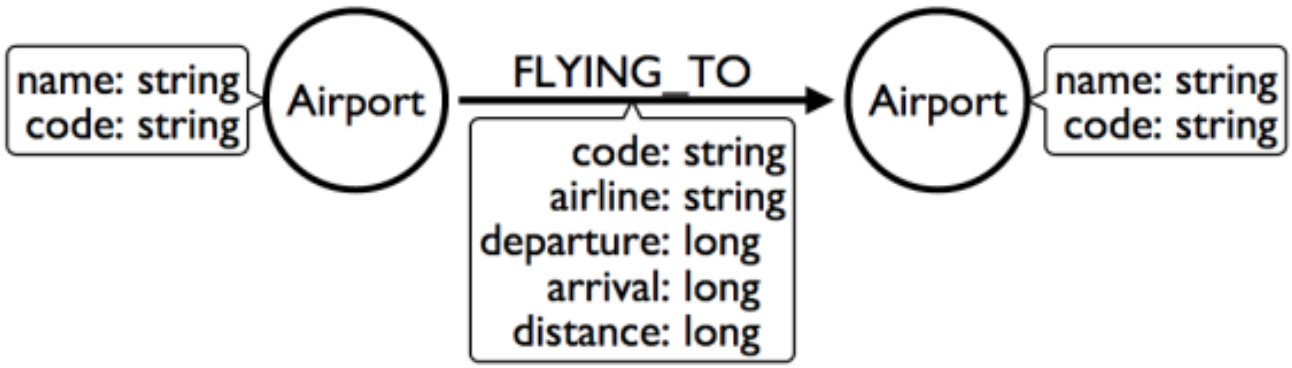
Ask yourself:

- What are the entities?
- What are the connections between the entities?
- What properties do we need?

Here, we begin by following the modeling process: identifying entities and connection based on the question. The entities should be Airports, and the connections FLYING_TO, with one connection per flight. Airport data like city and flight data like airline, flight number, departure, etc. are necessary properties.

Initial model

Question: What flights will take me from Malmo to New York on Friday?



This model answers the first question just fine. But suppose we had to answer a new question?

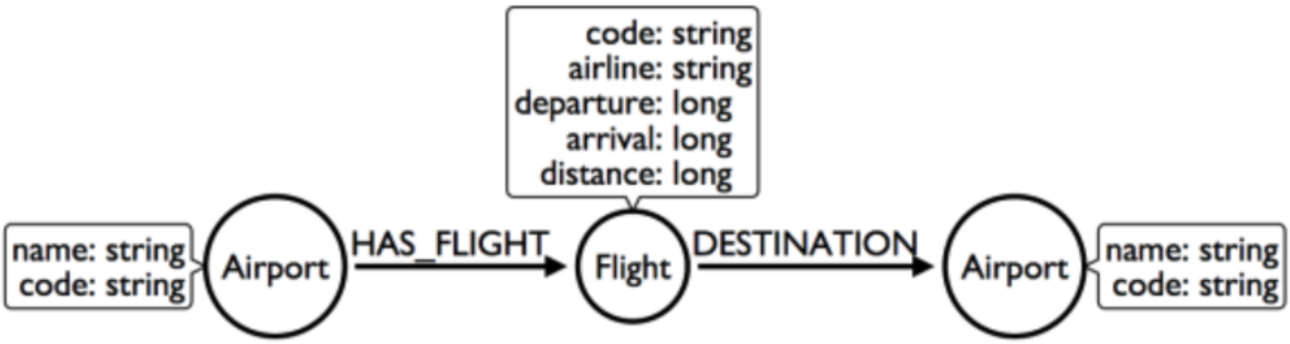
New Question: Mom is on flight AY189. When will she land?

What must Neo4j traverse to find this answer?

This is a painful one.To find flight AY189, we need to traverse every relationship in the graph, because it is impossible to anchor on relationships. What can we do to make flight data available as an anchor?

Refactor: Create intermediate Flight nodes

This is a perfect use case for adding intermediate nodes. Adding Flight nodes allows us to anchor on flight data, dramatically reducing traversal.



Question 1: What flights will take me from Malmo to New York on Friday?

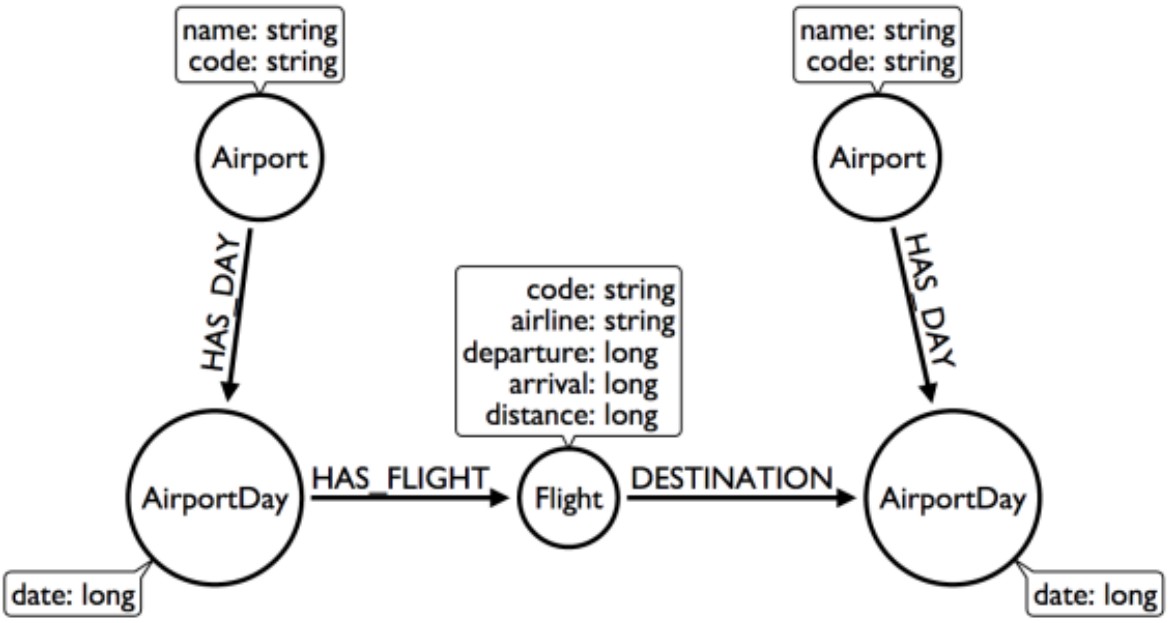
Question 2: Mom is on flight AY189. When will she land?

But we are still not satisfied with the way we are handling Q1. Airlines are required to publish flight plans 12 months or more in advance. How much work must Neo4j do to answer Q1?

Again, the answer is painful. Neo4j must check every flight leaving Malmo, then consult the flight data to see which ones leave on the appropriate day. That's even before we check to see which of those flights land in the desired place! How can we elevate the flight date in order to reduce the amount of wasted hops?

Refactor: Create AirportDay intermediate nodes

Again, intermediate nodes come to the rescue. AirportDay nodes reduce the density of Airport nodes, as there are many fewer days in the graph than there are flights. We still need to check every AirportDay to find the right date, but the scope of wasted traversal is lessened.



Question 1: What flights will take me from Malmo to New York on Friday?

Question 2: Mom is on flight AY189. When will she land?

Whenever we execute a model change, we also need to check that our older queries are not disrupted. What about Q2?

As long as we continue to be able to anchor on Flight, Q2 is unaffected. This refactor causes no problems.

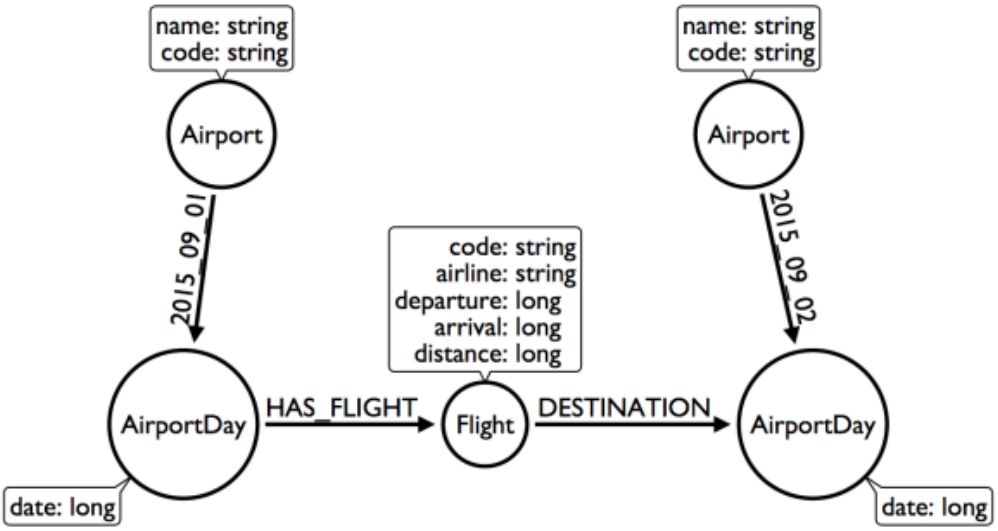
But thinking more on Q1. How might we arrange things to reduce wasted traversal even further? In other words, how might we elevate flight date even higher on the hierarchy of accessibility?

There are only two ways to do this:

- Anchor somehow on AirportDay.
- Make date into a relationship type.

We will refactor to make date a relationship type.

Possible refactor: Change relationship type to date



Question 1: What flights will take me from Malmo to New York on Friday?

Question 2: Mom is on flight AY189. When will she land?

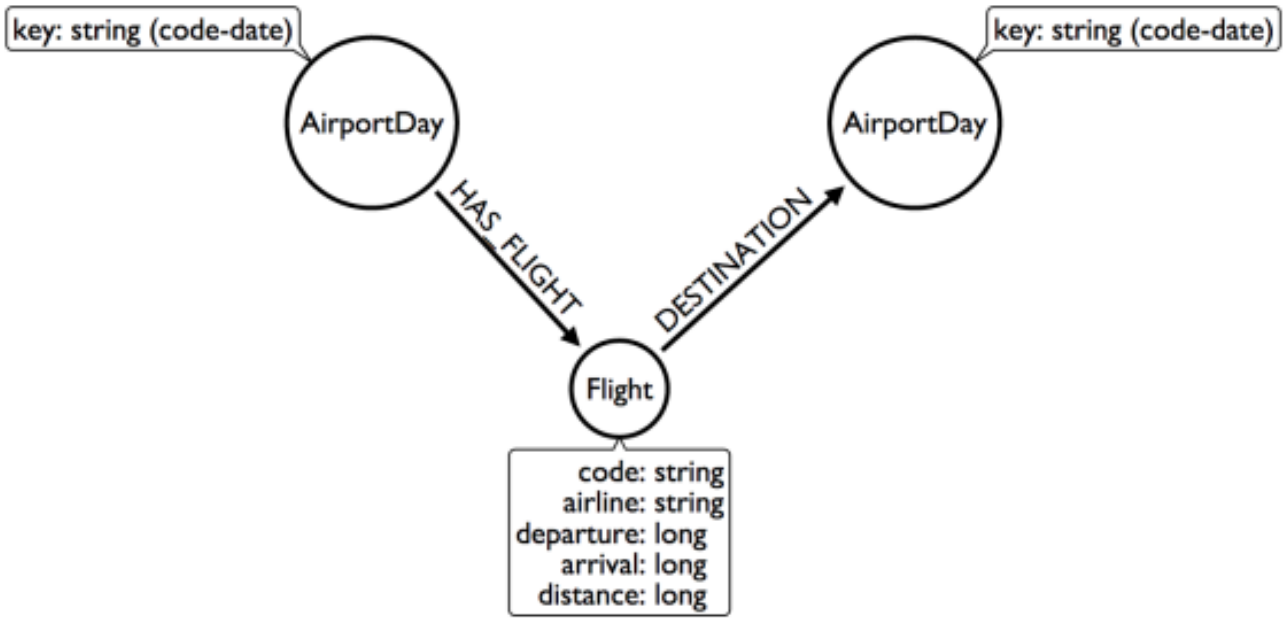
Date as the relationship type hardly changes the model at all, with drastic performance improvements. Now, we can traverse only to the relevant AirportDay. Again, Q2 is unaffected.

This is one of the two solutions we had. The other was to anchor on AirportDay. How would the model need to change to make that possible?

Possible refactor: Remove Airport nodes

We could eliminate the Airport nodes entirely, and store airport data on the AirportDay nodes. This shrinks the graph by removing a modest number of Airport nodes, and a large number of Date-

typed HAS_DAY relationships. This also performs 2 fewer hops per traversal—a tiny improvement, but one that could add up at scale. We would need to test this benefit more rigorously in production.



Question 1: What flights will take me from Malmo to New York on Friday?

Question 2: Mom is on flight AY189. When will she land?

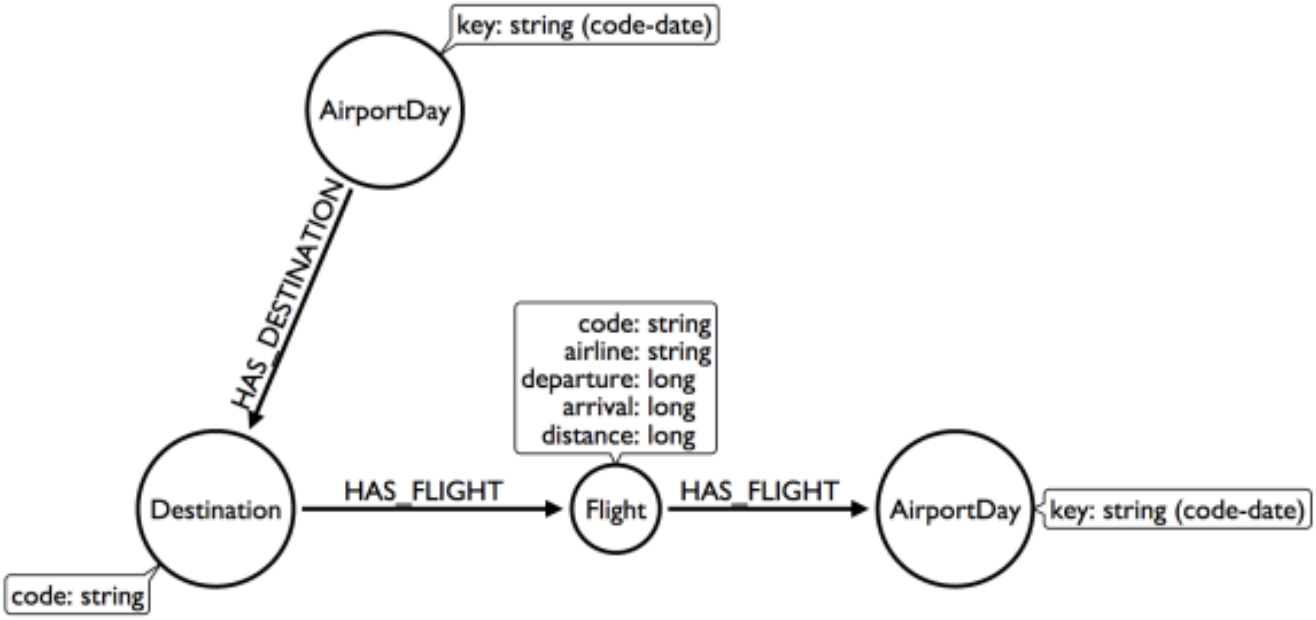
The downside to this model is that it is far less intuitive to a human looking at it. As mentioned earlier, refactoring is rarely pure positive; the gains you make in one area often involve sacrifices in others.

So we have a model that seems to work well for Q1. But we have been making one dangerous assumption: that there is a direct flight available. What will Neo4j need to do in order to find an itinerary with 2, 3, 4, or more legs?

This is an expensive problem for this model. Neo4j will traverse every flight leaving that day, and look at destinations. But in the non-direct case, none of the destinations is the correct one. So Neo4j will need to check all the HAS_FLIGHT relationships on those destinations, and see if the second-order destinations include the desired one. If that is not the case, repeat until you find the desired destination. The size of the traversed graph increases exponentially as more and more layovers are added. What can we do to reduce this?

Refactor: Add Destination intermediate nodes

Once again, intermediate nodes come to the rescue! In this case, we are doing two things, further subdividing flights based on destination, but more importantly, we are elevating flight destinations from a 3-hop downstream object to a 1-hop downstream object.



Question 1: What flights will take me from Malmo to New York on Friday?

Question 2: Mom is on flight AY189. When will she land?

How does the non-direct traversal work in this case?

First, Neo4j will check every Destination served by that AirportDay. If the target destination is not there, follow ONE Flight chain to the Airport Day of the destination, then check the served Destinations of that AirportDay. The scope of the graph still grows, but it grows at a rate proportional to the number of Destinations served by an airport, not the number of Flights. And airports tend to have multiple flights per destination, leaving at different times of day.

Once an itinerary leading to the target destination is found, Neo4j can branch out and traverse all the flights connecting those AirportDays.

As always, we must consider: how does this refactor affect Q2?

Once again, the answer is that it does not—we can still anchor on Flight, so Q2 is not disrupted.

Note that this model could never have been produced by simply following our “build initial model” paradigm. AirportDay and Destination nodes are completely opaque relative to the questions we asked. We are OK with that, because, as you will learn that when you implement the model in Cypher, refactoring a graph is relatively inexpensive. Moreover, getting a rough first model quickly reduces the total time we would need to reach this more refined version.

Check your understanding

Question 1

What tasks can be done during the refactoring of a graph data model?

Select the correct answers.

- ☑ Data is moved from one structure in the existing graph to another.

- ☐ A new graph is created from an existing graph.
- ☐ Statistics are collected about the numbers of nodes, properties, and relationships.
- ☑ Data may be added to the graph from other sources.

Question 2

Why do you refactor a graph data model?

Select the correct answers.

- ☑ Improve query performance.
- ☑ Simplify the model to make it more intuitive.
- ☑ Allow for simpler Cypher queries.
- ☑ Make updates to the data in the graph easier.

Question 3

When thinking about refactoring a graph data model. What is the most common type of refactoring you typically do?

Select the correct answer.

- ☐ Rename node labels.
- ☐ Duplicate property values where they will be queried most.
- ☑ Extract property values to change the structure of the graph.
- ☐ Create indexes that will speed up queries for the most important questions.

Summary

You should now be able to:

- Describe why you would refactor a graph data model.
- Refactor a model to:
  - Eliminate duplicate data in nodes.
  - Use node labels rather than properties.
  - Extract property values to create nodes.

Graph Data Modeling for Neo4j: Summary

Course Summary

In this course, you have learned how to:

- Describe what a graph data model is for a Neo4j database.
- Design an initial graph data model using Neo4j best practices.
- Describe the core principles used for Neo4j graph data modeling.
- Describe the common structures used in a Neo4j graph data model.
- Refactor and evolve a graph data model.

Resources

There are many resources available to you for learning more about Neo4j and Cypher.

https://neo4j.com/developer/resources/

Neo4j Community Site where you can ask or answer questions about Neo4j and discuss with other users:

https://community.neo4j.com

Neo4j documentation:

https://neo4j.com/docs/

Neo4j Sandboxes for experimenting with graphs:

https://sandbox.neo4j.com/?ref=graph-academy

Videos on the Neo4j YouTube channel:

https://www.youtube.com/channel/UCvze3hU6OZBkB1vkhH2lH9Q

Neo4j online and classroom training:

https://neo4j.com/graphacademy/

Become a Neo4j certified developer:

https://neo4j.com/graphacademy/neo4j-certification/

NOTE	You can take the certification exam multiple times until you pass!

GitHub repository:

https://github.com/neo4j-contrib

Neo4j events all over the world:

https://neo4j.com/events/world/all/

Graph Gists for learning more use cases for Neo4j:

https://neo4j.com/graphgists/

Attend a Neo4j meetup:

https://www.meetup.com/topics/neo4j/

View questions/answers raised about Neo4j:

https://stackoverflow.com/tags/neo4j/hot

Course feedback

We want your feedback on this course. Please provide your feedback so we can improve this course.